USER MANUAL FOR ZEUS ANCILLARIES

Version 1.0

David A. Clarke

Professor of astronomy and physics (retired)

Halifax NS, Canada

dclarke@zeus3d.ca

zeus3d.ca/zeus3d zeus3d.ca/azeus

December, 2015; modified 4/26

Copyright © David A. Clarke, 2026

Contents

Disclaimer **iii**

***PLOTZ* User Manual** **1**
1.1 Introduction, Version 1.3 . . . 1
1.2 Running *PLOTZ* . . . 1
1.2.1 Installation . . . 1
1.2.2 The script file `plotz13.s` . . . 2
1.2.3 A quick example using *PLOTZ* . . . 5
1.2.4 *PLOTZ* parameters . . . 7
1.3 Output from *PLOTZ* . . . 7
1.3.1 1-D plot files . . . 7
1.3.2 2-D plot files . . . 8
1.3.3 Message log files . . . 8
1.4 Namelists for *PLOTZ* . . . 8
1.4.1 `IOCON`—`I/O` `CON`trol (subroutine `IOINIT`) . . . 9
1.4.2 `INITCON`—`INIT`ialisation `CON`trol (subroutine `NMLSTS`) . . . 9
1.4.3 `PL1CON`—`PL`ot (`1`-D) `CON`trol (subroutine `NMLSTS`) . . . 10
1.4.4 `PL2CON`—`PL`ot (`2`-D) `CON`trol (subroutine `NMLSTS`) . . . 14

***RADIO* User Manual** **19**
2.1 Introduction, Version 1.3 . . . 19
2.2 Running *RADIO* . . . 19
2.2.1 Installation . . . 19
2.2.2 The script file `radio13.s` . . . 21
2.2.3 A quick example using *RADIO* . . . 24
2.2.4 *RADIO* parameters . . . 25
2.3 Output from *RADIO* . . . 25
2.3.1 1-D plot files . . . 26
2.3.2 2-D plot files . . . 26
2.3.3 Movie frames . . . 26
2.3.4 Message log files . . . 27
2.4 Namelists for *RADIO* . . . 27
2.4.1 `IOCON`—`I/O` `CON`trol (subroutine `IOINIT`) . . . 28
2.4.2 `INITCON`—`INIT`ialisation `CON`trol (subroutine `NMLSTS`) . . . 29
2.4.3 `VIEWCON`—`VIEW`ing `CON`trol (subroutine `NMLSTS`) . . . 29
2.4.4 `EMITCON`—`EMI`ssion `CON`trol (subroutine `NMLSTS`) . . . 30
2.4.5 `CONVCON`—`CONV`olution `CON`trol (subroutine `NMLSTS`) . . . 31
2.4.6 `MASKCON`—`MASK`ing `CON`trol (subroutine `NMLSTS`) . . . 31
2.4.7 `PL1CON`—`PL`ot (`1`-D) `CON`trol (subroutine `NMLSTS`) . . . 32
2.4.8 `PL2CON`—`PL`ot (`2`-D) `CON`trol (subroutine `NMLSTS`) . . . 34
2.4.9 `VECCON`—`VEC`tor `CON`trol (subroutine `NMLSTS`) . . . 36
2.4.10 `ANIMCON`—`ANIM`ation `CON`trol (subroutine `NMLSTS`) . . . 36

***ANIM8Z* User Manual** **39**
3.1 Introduction, Version 1.0 . . . 39
3.2 Running *ANIM8Z* . . . 39
3.2.1 Installation . . . 39
3.2.2 The script file `anim8z10.s` . . . 40
3.2.3 A quick example using *ANIM8Z* . . . 42
3.2.4 *ANIM8Z* parameters . . . 43
3.3 Output from *ANIM8Z* . . . 43
3.4 Namelist parameters for *ANIM8Z* . . . 43

***TESTZ* User Manual** **46**
4.1 Introduction, Version 1.0 . . . 46
4.2 Running *TESTZ* . . . 47
4.2.1 Installation . . . 47
4.2.2 The script file `testz10.s` . . . 49
4.2.3 Using *TESTZ* (`xtz10`) . . . 50
4.2.4 *TESTZ* parameters . . . 52
4.3 The test suite . . . 53
4.3.1 Advection tests . . . 53
4.3.2 Alfvén wave propagation . . . 54
4.3.3 Shock-tube tests . . . 54
4.3.4 Other 1-D tests . . . 55
4.3.5 Other 2-D tests . . . 56
4.3.6 3-D tests . . . 56
4.3.7 Adding tests to the test suite . . . 57
4.4 Output from *TESTZ* . . . 58
4.5 Namelist parameters for *TESTZ* . . . 58

Disclaimer

NOTICE: The software packages described in this manual have been developed by the author over the years starting in 1990 at the NCSA, U. Illinois and continuing through 2020 at the Institute for Computational Astrophysics at Saint Mary's University in Halifax, NS. They and this manual are offered "as is" by the author to anyone for non-profit, educational use with no expressed or implied warranty or suitability. It is requested that the author's name and this disclaimer remain associated with this manual and associated software, as well as any descendants of this software that may be developed by a third party.

1. *PLOTZ* User Manual

1.1 Introduction, Version 1.3

PLOTZ is a stand-alone program that encapsulates the 1-D and 2-D plotting capability of *ZEUS-3D*, and comes as part of the *ZEUS-3D* (version 3.6) public release tar-file `dzeus36.tar` (but not the abbreviated version `dz36.tar`). It takes one or more "total HDF files" generated by *ZEUS-3D* (of format `zhto`*nnn*`id`) as input and, using namelists `pl1con` and `pl2con` almost identical to those in *ZEUS-3D* (version 3.6), creates 1-D and 2-D plots in the same format as the in-line plotting of *ZEUS-3D*.

The executable for *PLOTZ* is created in an entirely analogous way as that for *ZEUS-3D*. The program is precompiled, compiled, loaded, and linked by use of the *script file* `plotz13.s` (described below), and is managed by the precompiler *EDITOR*. It is assumed the reader has already installed *ZEUS-3D* and *EDITOR* on their unix platform, is familiar with their use, and is looking for a way to visualise data that have been generated by a *ZEUS* run. Therefore, the reader who has not yet installed *ZEUS-3D* on their unix platform should do so first by downloading `dzeus36.tar` from zeus3d.ca/zeus3d and following the instructions found therein.

1.2 Running *PLOTZ*

1.2.1 Installation

This subsection describes how to install *PLOTZ* on your *UNIX*-based system. For a successful installation, HDF4 libraries must be available. NCAR libraries can be linked if desired, but are not necessary and therefore need not be available on the user's platform.

Having already downloaded `dzeus36.tar` from zeus3d.ca/zeus3d and unpacking the tarball, go to the directory `zeus_v3.6_ICA` and type `ls -FC`. On your screen you should see:

```
README          editor/         manuals/        radio/          zeus/
anim8z/         examples/       nmlst/          sci/
bldlib2*        grfx/           plotz/          testz/
```

In addition to the files in the directory `plotz/`, the *EDITOR* executable `editor/xedit22` and the libraries `nmlst/namelist.a`, `grfx/grfx03.a`, `grfx/psplot.a`, and `grfx/noncar.a` (all created during the installation of *ZEUS-3D*) are needed to create the *PLOTZ* executable, `xplotz13`.

Go to the `plotz` directory and once again type `ls -FC` where you'll find:

```
README.plotz    plotz13         plotz13.s       zhto002za
```

The file `plotz13` is the source code for *PLOTZ*, version 1.3, `plotz13.s` is the script file used to compile `plotz13` and described below in §1.2.2, and `zhto002za` is a data file generated by *ZEUS-3D* and used in the examples described in §1.2.3 and 2.2.3.

As is true for *ZEUS-3D* and *EDITOR*, *PLOTZ* specifically supports seven compilers: `cf77` (Cray), `f90` (Sun), `g95` (`g95.org`), `gfortran` (Gnu), `ifort` (Intel), `pgf77` (Portland Group), and `xlf` (IBM). If one of these compilers is available on the user's platform, go to step 4.

Step 1. If none of these compilers is available, then suppose the *FORTRAN* compiler that is available is called `newfor`. Open `plotz13` in a text editor, replace all occurrences of `GFORTRAN` (note all-caps) with `GFORTRAN.or.NEWFOR`, and save the changes.

Step 2. Open the file `plotz13.s` in a text editor, replace the one occurrence of `setenv FCOMP gfortran` with `setenv FCOMP newfor` (note no caps), and save the change.

Step 3. At your unix prompt, type: `csh -v plotz13.s`

If this succeeds in creating the executable `xplotz13`, installation is complete. If not, it will likely fail by citing calls to unknown implicit functions such as `cpu_time`, `date_and_time`, and/or `system` used by *PLOTZ* subroutines `RENAM`, `CCLOCK`, and `WCLOCK`. For many compilers such as `gfortran`, these implicit functions do the following:

`cpu_time`	returns the cpu time since the beginning of the run in seconds;
`date_and_time`	returns eight integers giving the year, month, day, *etc.* Search for `date_and_time` in `plotz13` for details.
`system`	executes a systems call (such as removing a disc file) at run time.

You will have to find the equivalents to these functionalities for `newfor`. Once determined, open up `plotz13` again, find the offending implicit functions, and make the necessary changes.

Step 4. At your unix prompt, type: `csh -v plotz13.s`

For those following steps 2 and 3, the complaints about the foreign system calls should disappear. If the executable `xplotz13` is created, installation is complete. If not, some troubleshooting will be necessary for which little general guidance can be given other than the discussion below on libraries (see paragraph beginning with: "Note that in addition to the "exposed"...).

Finally, at the end of the linking step, you may see warning messages of the type:

```
ld: warning: could not create compact unwind for _grfx2d_: stack subq
instruction is too different from dwarf stack size
```

I've no idea what these are, and no one on the "chat groups" seem to be able to explain them to me. What I do know is they seem to be safe to ignore.

1.2.2 The script file `plotz13.s`

The script file `plotz13.s` used in the previous subsection to install *PLOTZ* is similar to (and somewhat simpler than) the script file `dzeus36.s` used to install *ZEUS-3D*. As of this writing, the script file `plotz13.s` is as follows:

```
#---+----1----+----2----+----3----+--+----3----+----2----+----1----+---#
#============= SCRIPT FILE TO CREATE THE PLOTZ EXECUTABLE =============#
#
#---------------------------------------------> Select FORTRAN compiler.
#  Supported compilers include:
#     cf77 (Cray), f90 (Sun), g95 (g95.org), gfortran (Gnu), ifort
#     (Intel), pgf77 (Portland Group), xlf (IBM)
#
setenv FCOMP gfortran
#=====================================> Get files from home directory.
if(! -e ./xedit22) cp ../editor/xedit22 .
if(! -e ./namelist.a) cp ../nmlst/namelist.a .
if(! -e ./grfx03.a) cp ../grfx/grfx03.a .
if(! -e ./psplot.a) cp ../grfx/psplot.a .
if(! -e ./noncar.a) cp ../grfx/noncar.a .
#=======================> If necessary, create the directory "plotz1.3".
if(! -e ./plotz1.3) mkdir ./plotz1.3
#----------------------------------------------> Create the change deck.
rm -f ./chgplotz
cat << EOF > chgplotz
*define XYZ, MHD, ISO, $FCOMP
*d par.25,26
       parameter     (   in =  64,   jn =  64,   kn =  64 )
       parameter     ( niov =  20, nios =  20, ncls =  20 )
EOF
#=======================> Create the input deck for EDITOR, and execute.
rm -f ./inedit
cat << EOF > inedit
 \$editpar   inname='plotz13', chgdk='chgplotz', idump=1, job=3, ipre=1
           , inmlst=1, iupdate=1, iutask=0, ibanner=0, safety=0.50
           , branch='plotz1.3', xeq='xplotz13', makename='makeplotz'
           , compiler='$FCOMP'
c          , coptions='debug'
           , coptions='optimise'
           , libs='namelist.a grfx03.a psplot.a noncar.a
  -L/usr/local/gcc-fortran/lib -ldf -ljpeg -lz -lsz'                  \$
c          , libs='namelist.a grfx03.a psplot.a
c -L/usr/local/gcc-fortran/lib -ldf -ljpeg -lz -lsz -L/opt/local/lib
c -lncarg -lncarg_gks -lncarg_c -lX11 -lcairo -lfreetype'             \$
c          , libs='namelist.a grfx03.a psplot.a noncar.a'             \$
EOF
chmod 755 ./xedit22
./xedit22
#-----------------------------------> Create the input deck for PLOTZ.
rm -f ./inplotz
cat << EOF > inplotz
 \$iocon    infile='zhto002za'                                         \$
 \$initcon  nhy=111, prtime=0.1                                        \$
 \$pl1con   corl=1, aspect=1.0, npl1h=2, npl1v=2, norppl1=2
          , pl1dir=1,2,3, ipl1=1,1,1, jpl1=1,1,1, kpl1=1,1,1
          , pl1var='d ','v1','v2','v3','mA','b1','b2','b3'            \$
 \$pl2con   norppl2=2, pl2dir=3, lpl2=1
          , pl2sca1='d ','vn','bn','mA','vv'
          , pl2vec1='  ','v ','b ','  ','  '
          , pl2ng1=5*0, pl2ccol=5*2                                   \$
EOF
#==========================================> MAKE the PLOTZ executable.
make -f ./makeplotz
#=================================================> Tidy up directory.
```

```
 60  rm -f ./editlp ./inedit ./output ./xedit22 ./chgplotz ./chgplotz.m ./*.a
./plotz13.m ./plotz13.f
```

As with the *ZEUS-3D* script file `dzeus36.s`, the *PLOTZ* script file is divided into several segments heralded either by a double line (`====`) or a single line (`----`). Segments heralded by a double line are those which once set should rarely, if ever, be modified. Those heralded with a single line are segments that may have to be modified every time the program is used.

On line 9 of the first segment, the desired *FORTRAN* compiler is selected. Here, `FCOMP` is a "system variable" set by the *UNIX* `setenv` command, and is used in two places. First, on line 21 it serves as the *EDITOR* macro to tailor `plotz13` to the chosen compiler. Second, on line 32 it is the setting for the namelist parameter[1] `compiler` in the input deck `inedit` which determines which compiler the makefile uses to compile `plotz13`.

Lines 11–15 of the second segment retrieve the files necessary to create the *PLOTZ* executable and are retrieved only if they do not already exist in the present working directory (`pwd`) [`if (! -e` *filename*`)`]. This example assumes that the script file is launched from the directory `plotz` created when `dzeus36.tar` from zeus3d.ca/zeus3d is unpacked. Thus, the file `plotz13` (containing the source code) is already in this directory, and not retrieved. Files retrieved from other directories include:

`xedit22`	preprocessor executable
`namelist.a`	single precision library of subroutines to enable namelist feature
`grfx03.a`	library of graphics subroutines calling routines in external libraries (*NCAR* and *PSPLOT*)
`psplot.a`	library of routines for *PSPLOT* graphics
`noncar.a`	library of dummy *NCAR* routines used when *NCAR* graphics are not installed at the site

Line 17 of the third segment creates the directory `plotz1.3` into which all files split from the master source file `plotz13` by *EDITOR* and object files once compiled are placed.

The fourth segment (lines 19–25) generates the change deck `chgplotz` which gets merged with the master file `plotz13` by *EDITOR* during the precompilation step. Line 21 sets the various *EDITOR* macros necessary to tailor the compilation of *PLOTZ* (corresponding with the *EDITOR* macros used to compile *ZEUS-3D* that created the total HDF files *PLOTZ* will be expected to read), along with the compiler used to compile the program. Lines 22–24 set the array dimensions for the *ZEUS*-like variables stored in the HDF files. See §1.2.4 for further discussion.

The fifth segment (lines 27–43) generates the namelist input data file `inedit` which the *EDITOR* executable `xedit22` uses to precompile `plotz13`. In this example, the namelist parameters are set so that `chgplotz` will first be merged into `plotz13` which is then precompiled (since `ipre=1`) for `gfortran` with the namelist feature *on* (`inmlst=1`), the conditional

[1] The word "parameter" is, admittedly, a somewhat over-used term and the user needs to be aware of how it is being used by the context of the discussion. *FORTRAN* defines *parameters* to be immutable quantities used during the execution of a program that are set by `parameter` statements within the declaration segment of the source code. On the other hand, *namelist parameters* are, technically, variables as they could be changed during execution if need be. However, the term *namelist parameter* is so entrenched in the vernacular that it is difficult to avoid.

splitting feature *on* (`iupdate=1`), and the micro-tasking feature *off* (`iutask=0`). All files split from the merged master source code will be placed in the directory `plotz1.3` with the extension `.f`, appropriate for *FORTRAN* source code. Since `iupdate=1` *and* `makename` is set to a non-blank character string in the input deck, a makefile with the name `makeplotz` will be generated. The makefile will use the compiler `gfortran -c` and loader `gfortran` with the highest safe optimisation (`coptions='optimise'` not commented out). The specified libraries will be linked with the *PLOTZ* object code to generate a binary executable called `xplotz13`.

Note that in addition to the "exposed" list of libraries (lines 35–36) which assumes HDF4 libraries are available but not NCAR, the commented out list of libraries (lines 37–39) can be used if the NCAR libraries have been installed on the user's platform. As given, these lists are tailored for my own platform and will almost certainly fail on the user's platform. Indeed, it is with these lists that the new user may have to spend much time troubleshooting. Linking third-party libraries can be frustratingly tiresome, particularly as various installations and versions will have different library requirements, locations, *etc.*, and the user may well have to consult liberally with their systems administrator to get the right libraries in place and to make sure they are listed correctly in `plotz13.s`. The last library list (line 40) is one stripped of HDF4 and NCAR and thus should link without errors. However, without HDF4, *PLOTZ* is rather emasculated, as it will not be able to read any HDF dump files generated by *ZEUS-3D*.

Finally in this segment, line 43 launches `xedit22` and `plotz13` is precompiled.

The sixth segment (lines 45–56) sets the namelist input deck, `inplotz`, used by *PLOTZ* itself to generate the desired output from the specified input total HDF file(s). All namelist parameters for *PLOTZ* are described in §1.4.

Line 58 in the seventh segment fires up the makefile `makeplotz` generated in the fifth segment. The result of this step is the executable, `xplotz13`.

Finally, line 60 of the last segment removes some of the intermediate files no longer needed.

1.2.3 A quick example using *PLOTZ*

If `plotz13.s` executed successfully in the previous subsection, there should be an input deck `inplotz` (lines 47–55 in `plotz13.s` described above, without the backslashes before the $ signs) and an executable `xplotz13` in your `pwd`. These two files plus the data file `zhto002za` are all that are needed to run the example provided herein.

So, remaining within the `plotz` directory, type `xplotz13` at the unix prompt, and *PLOTZ* will fire up, making the plots as requested in `inplotz` from the total HDF dump `zhto002za`. As given, four separate plot files are produced. Three 1-D slices through the datacube are stored in files `pp002za.01.ps`, `pp002za.02.ps`, and `pp002za.03.ps`, and a single 2-D slice (with normal in the x_3-direction) is stored in the file `pq002za.01.ps`. See §1.3 for details on the plot file naming conventions. Each file has numerous plots, with the 1-D plots arranged four on a page and the 2-D plots one on a page.

Figure 1 shows on the left the first page of `pp002za.01.ps` and on the right the second page of `pq002za.01.ps`. The simulation is a very low-resolution (64^3) MHD simulation of a super-Alfvénic turbulent medium, one thought to model portions of the interstellar medium.

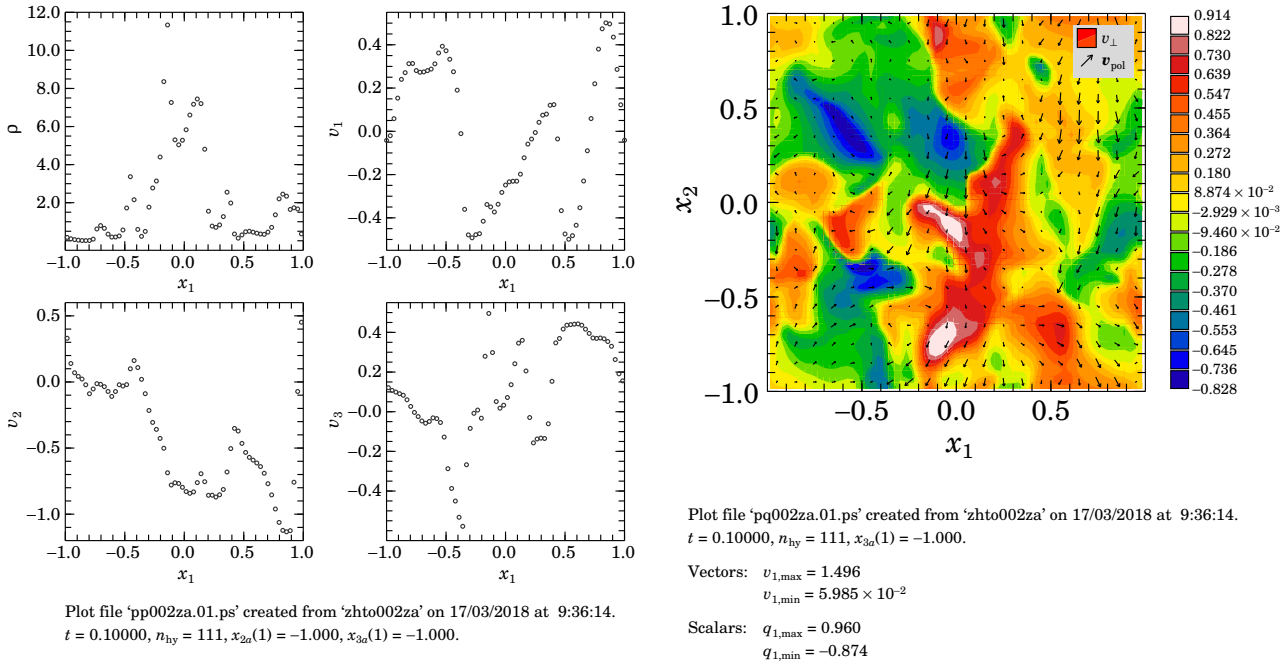


Figure 1: Left: Four 1-D plots extracted from the specified input HDF total file showing the density (top left) and the three components of velocity arranged as four panels on the same page using the 1-D plotting capability of *PLOTZ*. Right: A single 2-D plot of the normal velocity in colour contours with poloidal velocity vectors superposed using the 2-D plotting capability of *PLOTZ*. Note that *ZEUS-3D* uses the identical 1-D and 2-D plotting routines as *PLOTZ*.

Examining the input deck `inplotz` (essentially lines 47–55 in `plotz13.s` described above), the input file `zhto002za` is specified by the parameter `infile` in namelist `iocon`. The values of the problem time and number of cycles executed when the HDF dump was produced by *ZEUS-3D* are not data stored with the HDF file, and have to be included manually in the namelist `initcon` if these quantities are to be correct on the plot footers (which can be suppressed if desired: `pl1ftr=0`). The namelist `pl1con` specifies the user's choices for the 1-D plots, including whether plots are drawn with open circles (`corl=1`) or continuous lines (`corl=2`), that plots should be arranged two wide (`npl1h=2`) by two high (`npl1v=2`), that three separate slices should be generated, one parallel to each of the three axes of the datacube (`pl1dir=1,2,3`), and what variables should be plotted along each slice (`pl1var='d ',...`). Finally, namelist `pl2con` indicates what 2-D plots should be generated (certain variable combinations along one 2-D slice) and how they should appear (colour contours, vector lengths, *etc.*).

This example has touched on only a few of the namelist parameters available to control how *PLOTZ* functions. Note that most namelist parameters have sensible defaults, and thus need not be mentioned in the input deck `inplotz` to have an effect on the appearance of the plots. All namelist parameters for *PLOTZ* are described in detail in §1.4.

1.2.4 *PLOTZ* parameters

There are only a few parameters of which one must be aware when running *PLOTZ*, all familiar to *ZEUS-3D* users and all listed in lines 23–24 in the script file `plotz13.s` in §1.2.2. These are:

parameter	interpretation	default
in	number of active zones in 1-direction in ZEUS-3D run	
jn	number of active zones in 2-direction in ZEUS-3D run	
kn	number of active zones in 3-direction in ZEUS-3D run	
niov	maximum number of variables to be plotted	20
nios	maximum number of slices through datacube	20
ncls	maximum number of contour levels in 2-D plots	20

Note that `in`, `jn`, and `kn` should all be set to *five* less than they were in the *ZEUS-3D* simulation that generated the HDF dumps being read by *PLOTZ*. This is because HDF dumps do not store the ghost zones, only the active ones. The remaining three parameters need not be the same as they were in *ZEUS-3D*, but they may as well be.

1.3 Output from *PLOTZ*

In addition to a log-file, *PLOTZ* generates just two types of output files: 1-D and 2-D dumps in either *NCAR*'s metacode, or postscript generated by *PSPLOT*. Plots are all publication quality and virtually identical to those generated by *ZEUS-3D* at run time.

This section describes the characteristics of the output files including the namelist which controls the data dump (§1.4), the convention used for naming the file, and the format of the data in the file created.

1.3.1 1-D plot files

These are 1-D slices of any of a numerous selection of variables parallel to one of the coordinate axes. The data may be plotted continuously with smooth lines, or discontinuously with one of many available symbols specified by the user. A separate plot file containing slices of each of the variables requested is created for each slice specified.

namelist: `pl1con`

filename: `pp`*nnn*`id.`*mm*, where `pp` is the common prefix to all 1-D plot files generated by *PLOTZ*, *nnn* is the three-digit sequential number parsed from the input HDF file, `id` is the two-character ID tag also parsed from the HDF file, and *mm* is a two-digit extension indicating the slice number. For *PSPLOT*, the suffix `.ps` is added to the filename.

format: metacode—use `idt` to read *NCAR*-generated metafiles
postscript—use `mgv`/`gv` to read *PSPLOT*-generated postscript files

These files are similar in every way to the *ZEUS*-created files `zp`*nnn*`id.`*mm*(`.ps`).

1.3.2 2-D plot files

2-D plots are created using the "buffet-style plotting" introduced to version 3.6 of *ZEUS-3D* (`dzeus36`). In this method, the user selects as many as two scalars (one plotted as colour-filled contours, the other as superposed contour lines), with as many as three vector fields (using arrows of different colours) in whatever combination the user may want (*e.g.*, density colour contours with magnetic field lines superposed, along with vectors representing velocity, pressure gradient, and the Lorentz force). A separate plot file containing slices of each of the variable combinations requested is created for each slice specified.

Note that *PLOTZ* does not have access to the data files containing the locations and properties of the Lagrangian particles, and thus the "corks" feature of *ZEUS-3D* is not supported by `plotz13`; see §1.4.4 for further details.

namelist: `pl2con`
filename: `pq`*nnn*`id`.*mm*, where `pq` is the common prefix to all 2-D plot files generated by *PLOTZ*, *nnn*, `id`, and *mm* are as described for 1-D plots. For *PSPLOT*, the suffix `.ps` is added to the filename.
format: metacode—use `idt` to read *NCAR*-generated metafiles
postscript—use `mgv`/`gv` to read *PSPLOT*-generated postscript files

These files are similar in every way to the *ZEUS*-created files `zq`*nnn*`id.`*mm*(`.ps`).

1.3.3 Message log files

The message log file contains all the messages that are written to the terminal during execution, and serves as a record of the run. This feature is inherited from *ZEUS-3D*, where such files are more useful. For *PLOTZ*, these files are largely vestigial.

namelist: `none`
filename: `pl`*nnn*`id`, where `pl` is the common prefix to all log files generated by *PLOTZ*, *nnn* and `id` are as defined for 1-D plot files.
format: ascii

1.4 Namelists for *PLOTZ*

Most of the `namelist` parameters used to specify the plots required from the input HDF files are identical to those in *ZEUS-3D*. Indeed, one may copy-paste the namelists `pl1con` and `pl2con` right from the *ZEUS* input deck `inzeus` to `inplotz` and get the same plot as *ZEUS* would have produced from the data written to the current HDF total file.

This section gives a complete catalogue of all the namelist parameters in `plotz13`. As with the *ZEUS* manual, all parameters are grouped together in separate namelists and discussion for each namelist is contained within a subsection headed by the name of the namelist and the `plotz13` subroutine in which the namelist is called. After each heading is a discussion of what the namelist controls, a list of all the parameters which are elements of the namelist, and finally the syntax used in `plotz13` to declare the namelist. For a reminder

of the nature of namelists and how they are used in the *ZEUS* family of codes, the reader is referred to the first page of Appendix B in the `dzeus36` user manual.

1.4.1 `IOCON`—`I/O CON`trol (subroutine `IOINIT`)

This namelist controls which total HDF files are read by `plotz13` during execution. One can specify a single file (`infile`) or, if multiple files are to be read all of which are to have the same plots extracted, one can further specify `lastfile` in the sequence to be read (`infile` being the first). Here, the usual format for *ZEUS* HDF files (`zhto`*nnn*`id`) is assumed, and both `infile` and `lastfile` must have the same two-character `id`. The routine starts with `infile`, adds `mdinc` to *nnn* and reads that file, *etc.* until `lastfile` is read. If any file along the way is not found, execution is aborted.

```
parameter                    description                        default

infile          character*32 name of first input HDF file to
                read.  The 32 characters allow for a modest
                directory path to precede filename, if desired.
lastfile        character*32 name of last input HDF file to read
zora            =1 => infile and lastfile are ZEUS-3D files of       1
                      form: zhtoNNNID, where "NNN" is a 3-digit
                      sequential number and "ID" is a two-
                      character problem tag
                =2 => infile and lastfile are AZEuS files of
                      form: ahtoNNNID.GG, where "NNN" and "ID are
                      as above, "GG" is two-digit grid identifier
mdinc           increment between files.  0 => read only "infile"    0
                e.g., if every 3rd file zhto001aa --> zhto100aa
                is to be read, set mdinc=3.  If every file is to
                be read, set mdinc=1.

       namelist / iocon    /
     1               infile  , lastfile, zora    , mdinc
```

1.4.2 `INITCON`—`INIT`ialisation `CON`trol (subroutine `NMLSTS`)

This namelist includes the relevant *ZEUS*-parameters which help characterise the simulation that were not stored with the HDF file. These will be needed, for example, if the pressure has to be calculated from the internal energy (requiring the adiabatic index), if a synchrotron or bremsstrahlung emissivity has to be computed, or if any point source gravitational masses are required (for reproducing the components of the gravitational potential other than the self-gravitational term). Further, all plots are labelled with the problem time (`prtime`) and the number of cycles of execution (`nhy`) corresponding to when the HDF file was created, and these too have to be input.

```
parameter                    description                        default

gamma   (   2)  adiabatic index of two fluids                     5/3
ciso    (   2)  isothermal sound speed of two fluids              1.0
prtime          problem time of HDF dump; used to label graphs
nhy             cycle of HDF dump; used to label graphs
brn0            fiducial number density for bremsstrahlung        lrge
```

```
                emission (m**-3)
brt0            fiducial temperature for brem. emission (K)         lrge
brnu1,brnu2     limits of frequency band for Brem. emission (Hz)    1.0000e17,
                                                                    1.0001e17
radtype         0 => emissivities are not computed                  2
                1 => Smith et al emissivity (p**2)
                2 => CNB emissivity (function of d, p, B)
                3 => CNB emissivity, including break freq.
pindex          power law index of relativistic electrons           2.0
                N(E) = kappa * E**( -pindex ) (radtype=2,3 only)
theta0          desired angle between x1-axis and line of sight    90.0
                for synchrotron emissivity (degrees).
phi0            desired azimuthal angle about x1-axis for           0.0
                synchrotron emissivity (degrees).
freq            frequency of RADIO observation (Hz)                 5.0e9
nubrs           nubr(Hz)/nubr(zeus); scaling factor for break       0.5
                frequency (radtype=3 only)
nu              kinematic viscosity (same as used in the
                corresponding ZEUS run, and needed for plots of
                kinematic viscous transport (plot1d).
gcnst           gravitational constant                              0.25/pi
                = 0.25/pi for unitless calculations
                = 6.67259e-11 for mks
                = 6.67259e-08 for cgs
ptmass          fixed central point mass object.  For scaled        0.0
                units, "ptmass" (scaled point mass M) depends
                on "gcnst", scaling for density, and grid size.
                For gcnst = 1/(4pi), ptmass = (4 pi GM)/(ds rs**3),
                where "ds" is density scale and "rs" is length
                scale.  For M = 1 solar mass, ds = 3.0e5 H atoms
                per m**3, and rs = 1.0e3 AU, ptmass ~ 1.
iptmass         i index of point mass                               is
jptmass         j index of point mass                               js
kptmass         k index of point mass                               ks

       namelist / initcon  /
     1               gamma   , ciso    , prtime  , nhy     , brn0
     2             , brt0    , brnu1   , brnu2   , radtype , pindex
     3             , theta0  , phi0    , freq    , nubrs   , nu
     4             , gcnst   , ptmass  , iptmass , jptmass , kptmass
```

1.4.3 PL1CON—PLot (1-D) CONtrol (subroutine NMLSTS)

This namelist controls the 1-D graphics. For each HDF file read, as many as `nios` 1-D slices may be selected, each of which may specify as many as `niov` variables, where `nios` and `niov` are parameter set before compilation (§1.2.4). For every slice chosen, a file (in either metacode or postscript) is created with a separate plot generated for each variable specified. These plots may be arranged in the same frame or separate frames, and can have any rectangular shape desired. All plots are of publication quality. Each 1-D slice (bounded by `x1p1mn`, `x1pl1mx`, *etc.*) runs parallel to one of the axes of the computational grid. To specify the slice uniquely, two of `ipl1`, `jpl1`, and `kpl1` must be set. If `ipl1mn`, *etc.* is 0, `x1pl1mn`, *etc.* is used instead.

Several parameters in this namelist are inherited from `dzeus36` and can be set to anything without having any effect on the plots. These are carried over simply so that the `pl1con` from

the `dzeus36` input file `inzeus` can be copy-pasted directly to `plotz13` input file `inplotz` without causing "unknown namelist parameter" errors on execution. These include `dtpl1`, `npl1dmp`, `pl1soln`, `xdiscpl1`, and `dtgr1d`.

```
parameter                    description                        default

General

dtpl1           vestige from dzeus36; has no effect here.
npl1dmp         vestige from dzeus36; has no effect here.
units           sets the units of angular dimensions (character*2)   'rd'
                'dg' => degrees, 'rd' => radians
                'pi' => units of pi radians
aspect          < 1.0 => plots are wide and short                    1.0
                = 1.0 => square plots
                > 1.0 => plots are tall and narrow
pwtpl1          pen weight for 1-D plots                             1.0
norppl1         = 1 => use NCAR graphics library for 1-D plots       2
                = 2 => use PSPLOT graphics library for 1-D plots
corl            selector for lines and symbols for plots             1
                =-6 => open diamonds; one per plot element
                =-5 => open pentagrams
                =-4 => asterisk (*)
                =-3 => open squares
                =-2 => open triangles
                =-1 => plus signs (+)
                = 0 => crosses (x)
                = 1 => open circles (o)
                = 2 => smooth curve drawn through points
                any other value => no plot
npl1h           number of frames horizontally per page               1
npl1v           number of frames vertically   per page               1
allxl           = 2 => all x labels/annotations are drawn            2
                = 1 => only the x-label/annotations on the bottom
                       row of plots on the page are drawn.
                = 0 => labels aren't even drawn on the bottom row.
                allxl < 0 same as abs(allxl) except last annotation
                is not drawn if it falls at right edge of frame.
                If plots are assembled a posterori, this is useful
                to prevent right-most annotation of left plot from
                overwriting left-most annotation of right plot.
allyl           = 2 => all y labels/annotations are drawn            2
                = 1 => only the y-label/annotations on the left
                       column of plots on the page are drawn.
                = 0 => labels aren't even drawn on the left column.
pl1hdr          = 0 => suppresses header                             1
                = 1 => header is printed on top    of plot
pl1ftr          = 0 => suppresses footer                             1
                = 1 => footer is printed on bottom of plot

Slice control

pl1dir  (nios)  direction of 1-D plot: 1, 2, or 3 => 1-, 2-, or      0
                3-direction.  0 => no plots.
ipl1    (nios)  i index of 1-D plot in 2- or 3-direction        (is+ie)/2
jpl1    (nios)  j index of 1-D plot in 3- or 1-direction        (js+je)/2
kpl1    (nios)  k index of 1-D plot in 1- or 2-direction        (ks+ke)/2
x1pl1mn (nios)  minimum x1 of slice; or used by "pl1mean"       x1a(is)
```

```
x1pl1mx (nios)  maximum x1 of slice; or used by "pl1mean"       x1a(ie+1)
x2pl1mn (nios)  minimum x2 of slice; or used by "pl1mean"       x2a(js)
x2pl1mx (nios)  maximum x2 of slice; or used by "pl1mean"       x2a(je+1)
x3pl1mn (nios)  minimum x3 of slice; or used by "pl1mean"       x3a(ks)
x3pl1mx (nios)  maximum x3 of slice; or used by "pl1mean"       x3a(ke+1)
ipl1mn  (nios)  i-index of minimum x1 of slice                       0
ipl1mx  (nios)  i-index of maximum x1 of slice                       0
jpl1mn  (nios)  j-index of minimum x2 of slice                       0
jpl1mx  (nios)  j-index of maximum x2 of slice                       0
kpl1mn  (nios)  k-index of minimum x3 of slice                       0
kpl1mx  (nios)  k-index of maximum x3 of slice                       0
pl1xlab (nios)  character string containing NCAR mark-up language
                (see below) for the desired x-label.  Default is
                the generic coordinate label (x1, x2, x3).

NCAR markup syntax:

    Fonts: |F0|  = times-roman (default)
           |F10| = times-roman italic
           |F14| = times-roman italic bold
           |F5|  = greek italic: |F5|r => \rho; |F5|w => \omega
           |F18| = math characters: |F18|A = \perp; |F18|R => \parallel
                                     |F18|Q = | (vertical bar)
           |F34| = math characters: |F34|W = \cdot; |F34|Q => \nabla
                                     |F34|0 = \deg

    Position: |B| => subscript; |S| => superscript; |N| => normal

    One may combine one position designator with one font change in
    each "field" (bounded by ||), and then string as many fields
    together as one needs:
           "|F10|e|BF0|t|NF10|+e|BF0|r" = $e_{\rm t}+e_{\rm r}$

Variable control

pl1var  (niov)  names of variables to be plotted (character*2).      'zz'
                Valid names are: 'a ' (vector potential norm),
                'a1', 'a2', 'a3' (vector potential components),
                'ag' (synchrotron age), 'B3' (b3*b3), 'b ' (mag-
                netic field norm), 'b1', 'b2', 'b3', 'ba', 'bb',
                'bc' (magnetic field components), 'bd' (magnetic
                field norm/density), 'bP', 'bR' (phi- and radial
                magnetic field components), 'br' (bremsstrahlung
                emissivity), 'bt' (plasma beta), 'cs' (sound
                speed), 'd ' (density), 'da' (density-age), 'e1'
                (1st int. energy), 'e2' (2nd int. energy), 'er'
                (radiation energy), 'et' (total energy density),
                'gl' [d(r lt) / dr], 'gk' [d(r kt) / dr], 'gp'
                (gravitational potential), 'hy' [(1/rho) dp/dr -
                3/(2r**2)], 'j ' (current density norm), 'j1',
                'j2', 'j3' (current density components), 'k1'
                (entropy of 1st fluid), 'k2' (2nd entropy), 'ka'
                (avg. 1st + 2nd ent.), 'kt' (kinetic viscous
                radial trans.), 'lu' (radiative luminosity), 'l '
                (angular momentum), 'lt' (angular momentum radial
                transport), 'm ' (Mach number), 'mA' (Alfven Mach
                number), 'mf' (fast MS Mach number), 'ms' (slow
                MS Mach number), 'om' (angular velocity), 'p1'
                (1st thermal pressure), 'p2' (magnetic pressure),
```

```
                 'p3' (1st ther. + mag. pres.), 'p4' (2nd thermal
                 pressure), 'p5' (1st ther. + 2nd ther. pres.),
                 'p6' (mag. + 2nd ther. pres.), 'p7' (1st ther. +
                 mag. + 2nd ther. pres.), 'pa' [pitch angle;
                 atan(b1/bP)], 'pg' (pseudo-gravitational poten-
                 tial), 'ps' [atan(b3/b2)], 's1', 's2', 's3'
                 (momentum components), 'sy' (synchrotron emissiv-
                 ity), 'u1' (1st specific int. energy), 'u2' (2nd
                 specific int. energy), 'v ' (speed), 'v1', 'v2',
                 'v3', 'va', 'vb', 'vc' (velocity components),
                 'vA' (Alfven speed), 'vf' (fast speed), 'vs'
                 (slow speed), 'vv' [div(v)], 'w ' (vorticity
                 norm), 'w1', 'w2', 'w3' (vorticity components).
nlpl1   (niov)   = 1 => linear-linear     plot                        1
                 = 2 => log(y)-linear(x)  plot
                 = 3 => linear(y)-log(x)  plot
                 = 4 => log-log           plot
pl1min  (niov)   minimum value to be plotted.                        0.0
pl1max  (niov)   maximum value to be plotted.                        0.0
pl1mm            = 1 => "pl1min" and "pl1max" are set to extrema     1
                        of each variable.
                 = 0 => Use input values of "pl1min" & "pl1max" to
                        set range of plot.  If pl1max .le. pl1min,
                        they are set to extrema (as though pl1mm=1)
                 =-1 => same as 0, but extend plot range by one
                        minor tick mark in each direction
pl1mean (niov)   = 0 => ordinary 1-D slices; no means taken.         0
                 = 1 => 1-D slices are filled with means of
                        variable across plane (limited by x*pl1mn
                        and x*pl1mx) orthogonal to the slice.
thetapl1         meridonal rotational angle between 1-2-3 & a-b-c    0.0
                 coordinates (to compute va vb vc from v1 v2 v3)
phipl1           azimuthal rotational angle between 1-2-3 & a-b-c    0.0
                 coordinates (to compute va vb vc from v1 v2 v3)
pl1ylab (niov)   character string containing NCAR mark-up language
                 for the desired y-label.  Most defaults are fine,
                 unless, for e.g., v_\phi is desired over v_3.

Overlays

pl1grid          every (pl1grid)th a-grid line is drawn as a fine    0
                 vertical line (for both regular and diagn. plots)
                 = 0 => no grid lines are drawn
pl1ref  (niov)   dashed reference line drawn on plot                -huge
                 (< -1.0d30 => no line)
pl1soln (niov)   vestige from dzeus36; has no effect here.
xdiscpl1(nios)   vestige from dzeus36; has no effect here.

Diagnostic dumps

dtgr1d           vestige from dzeus36; has no effect here.
```

If you are using *NCAR* graphics (`norppl1=1`), you will need to link all *NCAR* graphics libraries to your executable (see your system administrator if you do not know what or where these libraries are) as well as two user-created libraries, `grfx03.a` and `psplot.a`. If you are using *PSPLOT* (`norppl1=2`) exclusively, you can dispense with all the *NCAR* libraries, and just link `grfx03.a`, `psplot.a`, and `noncar.a`, the last of which serves only to satisfy external

calls to non-existing *NCAR* subroutines.

```
  namelist / pl1con   /
1              dtpl1   , npl1dmp , units   , aspect  , pwtpl1
2            , norppl1 , corl    , npl1h   , npl1v   , allxl
3            , allyl   , pl1hdr  , pl1ftr  , pl1dir  , ipl1
4            , jpl1    , kpl1    , x1pl1mn , x1pl1mx , x2pl1mn
5            , x2pl1mx , x3pl1mn , x3pl1mx , ipl1mn  , ipl1mx
6            , jpl1mn  , jpl1mx  , kpl1mn  , kpl1mx  , pl1xlab
7            , pl1var  , nlpl1   , pl1min  , pl1max  , pl1mm
8            , pl1mean , thetapl1, phipl1  , pl1ylab , pl1grid
9            , pl1ref  , pl1soln , xdiscpl1, dtgr1d
```

1.4.4 PL2CON—PLot (2-D) CONtrol (subroutine NMLSTS)

This namelist controls the 2-D graphics. For each HDF file read, as many as `nios` 2-D slices may be specified for as many as `niov` variable combinations to be plotted. For every slice chosen, a file (metacode or postscript) is created with a plot generated for each individual or combination of variables specified. The normal to each slice is parallel to one of the axes of the computational grid and is specified by `pl2dir`. The extent of the slice is limited by `x1pl2mn`, `x1pl2mx`, *etc.*, while the index at the base of the normal to the slice is given by `lpl2`. If `ipl2mn`, *etc.* is 0, `x1pl2mn`, *etc.* is used instead.

These plots are created using the *buffet-style plotting* introduced into `dzeus36` where the user may select for each plot as many as two scalars and three vector fields. The Lagrangian particle feature (corks) also introduced in `dzeus36` has not been exported to `plotz13`, and should not be used here. Attempting to define CORKS as an *EDITOR* macro will cause compilation-time syntax errors since most of the cork-related variables have not been properly declared.

For *PSPLOT* graphics, the first scalar may be plotted using either colour or line contours while the second scalar may be plotted using line contours only. Each vector field may be drawn with different colours and a legend is drawn to associate each scalar and vector with the chosen variable. For *NCAR* graphics, both scalars must be line contours, and only one vector field may be specified.

PSPLOT 2-D graphics can generate *enormous* post-script files. A single file with several variables plotted in full colour from a high-resolution data-slice (*e.g.*, 1,000 zones on a side) can approach 1 Gbyte. Not only does this needlessly occupy disc space and cause commonly-available graphics-viewers such as `ghostview` to take forever opening each page, it can also soak up all the available memory on the machine while the plots are being generated, which is particularly intolerable when running *PSPLOT* in-line with *ZEUS-3D*. Such plots tend to be *hyper-resolved* often with multiple graphics instructions assigned to each screen pixel, and are rarely worth the computational effort. One can mitigate against this by specifying parameters `npl2i` and `npl2j` to be smaller than the actual grid dimensions (*e.g.*, of the order 250–500). These parameters set the dimensions of the *re-binning* grids into which the 2-D data-slices are rebinned (much like pixel plots in *ZEUS-3D*) before plotting. This can decimate the post-script file while still generating perfectly acceptable plots, qualitatively identical to the hyper-resolved ones.

All plots are of publication quality and come fully labelled, including a time stamp for easy identification. Unlike the 1-D plots, only one plot may be written to each frame.

However, the plot may be scaled down (`pl2scale`) if desired.

```
parameter                    description                        default

General

dtpl2           vestige from dzeus36; has no effect here.
npl2dmp         vestige from dzeus36; has no effect here.
units           sets units of angular dimensions (character*2).      'zz'
                'dg' => degrees, 'rd' => radians,
                'pi' => units of pi radians
pl2scale        .lt. 1.0d0 => scaling factor to reduce plot size     1.0
                .ge. 1.0d0 => full size
pwtpl2          pen weight for 2-D plots                             1.0
norppl2         = 1 => use NCAR graphics library for 2-D plots       2
                = 2 => use PSPLOT graphics library for 2-D plots
pl2grid         = 0 => do not overlay grid lines onto plots          0
                > 0 => overlay a-grid
                < 0 => overlay b grid
                every mod(|pl2grid|,1000)th grid line drawn

Slice control

pl2dir  (nios)  direction of normal to image plane: 1, 2, or 3 =>    0
                1-, 2-, or 3-direction.  0 => no plots.
lpl2    (nios)  level of 2-D plot (value of 1-, 2-, or 3-index) (is+ie)/2
pl2avg  (nios)  = 1 => averages slice with lpl2-1
                = 0 => no average taken (default)
x1pl2mn (nios)  minimum x1 of plot window                       x1a(ismn  )
x1pl2mx (nios)  maximum x1 of plot window                       x1a(iemxp1)
x2pl2mn (nios)  minimum x2 of plot window                       x2a(jsmn  )
x2pl2mx (nios)  maximum x2 of plot window                       x2a(jemxp1)
x3pl2mn (nios)  minimum x3 of plot window                       x3a(ksmn  )
x3pl2mx (nios)  maximum x3 of plot window                       x3a(kemxp1)
ipl2mn  (nios)  i-index of minimum x1 of plot window                 0
ipl2mx  (nios)  i-index of maximum x1 of plot window                 0
jpl2mn  (nios)  j-index of minimum x2 of plot window                 0
jpl2mx  (nios)  j-index of maximum x2 of plot window                 0
kpl2mn  (nios)  k-index of minimum x3 of plot window                 0
kpl2mx  (nios)  k-index of maximum x3 of plot window                 0
iflippl2(nios)  = 0 => no flipping of plots                          0
                = 1 => plot flipped about x-axis before writing
jflippl2(nios)  = 0 => plot not flipped about x-axis                 0
                = 1 => plot flipped about y-axis before writing
ireflpl2(nios)  = 0 => plot not reflected about x-axis               0
                = 1 => plot reflected about x-axis before writing
                       rendering plot twice as tall
jreflpl2(nios)  = 0 => plot not reflected about y-axis               0
                = 1 => plot reflected about y-axis before writing
                       rendering plot twice as long
npl2i   (nios)  no. of horizontal zones in rebinned grid (<= idim)   0
                = 0 => no rebinning in x- or y-direction
npl2j   (nios)  no. of vertical   zones in rebinned grid (<= jdim)   0
                = 0 => no rebinning in x- or y-direction
pl2xlab (nios)  character string containing NCAR mark-up language
pl2ylab (nios)  for the desired x- and y-labels.  Default is the
                generic coordinate label (x1, x2, x3).
pl2xl   (nios)  = 0 => suppresses xlabel (but not x-annotations)     1
                = 1 => xlabel is put below horizontal axis
```

```
pl2yl   (nios)  = 0 => suppresses ylabel (but not y-annotations)     1
                = 1 => ylabel is put beside vertical  axis
pl2ftr  (nios)  = 0 => suppresses footer                             1
                = 1 => footer is printed on bottom of plot
pl2leg  (nios)  = 0 => suppresses legend                             2
                = 1 => legend is printed above plot (as a header)
                = 2 => legend is printed on inside top right
                       corner of plot

If "ipl2mn", etc. is 0, "x1pl2mn", etc. is used instead.

Contours; for contour variables, * = 1,2

pl2sca* (niov)  names of scalars to plot (character*2)               'zz'
                Valid names are: 'a ', 'a1', 'a2', 'a3', 'ag',
                'an' (normal vector potential), 'b ', 'b1', 'b2',
                'b3', 'bn' (normal magnetic field), 'bP', 'bR',
                'bt', 'cs', 'd ', 'da', 'e1', 'e2', 'er', 'et',
                'fn' (normal magnetic flux function), 'gp', 'j ',
                'j1', 'j2', 'j3', 'jn' (normal current density),
                'k1', 'k2', 'ka', 'lu', 'm ', 'mA', 'mf', 'ms',
                'nb' (break frequency), 'p1', 'p2', 'p3', 'p4',
                'p5', 'p6', 'p7', 'pa', 'pg', 's1', 's2', 's3',
                's ' (momentum norm), 'sn' (normal momentum),
                'sy', 'u1', 'u2', 'v ', 'v1', 'v2', 'v3', 'vA',
                'vf', 'vn' (normal velocity), 'vs', 'vv', 'w ',
                'w1', 'w2', 'w3', 'wn' (normal vorticity).
                See "pl1var" in pl1con for other definitions.
pl2ng*  (niov)  = 1 => contours are spaced evenly                    1
                > 1 => contours are spaced geometrically such
                       that first two contours are spaced "pl2ng*"
                       times closer than evenly spaced contours.
                <-1 => contours are spaced geometrically such
                       that last two contours are spaced "-pl2ng*"
                       times closer than evenly spaced contours.
pl2min* (niov)  minimum contour level to be plotted.                 0.0
pl2max* (niov)  maximum contour level to be plotted.                 0.0
pl2mm*          = 0 => use input "pl2min*" & "pl2max*" for plots     1
                = 1 => compute "pl2min*" & "pl2max*" for plots
                If both "pl2min*" and "pl2max*" are 0.0, compute
                them as though "pl2mm*" were 1 for that variable.
thetapl2        meridonal rotation angle between 1-2-3 and a-b-c     0.0
                coordinates (compute va, vb, vc from v1, v2, v3)
phipl2          azimuthal rotation angle between 1-2-3 and a-b-c     0.0
                coordinates (compute va, vb, vc from v1, v2, v3)
numcl*  (niov)  = 0 => min(20,ncls) evenly spaced contours are       0
                      drawn between "pl2min*" and "pl2max*"
                > 0 => number of evenly spaced contours to draw
                       between "pl2min*" and "pl2max*" (max ncls).
                < 0 => abs(numcl*) contours drawn and are
                       specified by "pl2min*" and "levs*".
                if numcl*(2:niov) are not set by user, they are
                set to numcl*(1).
levs*   (ncls)  real array specifying multiples of "pl2min*" to
                use for contour levels (for numcl* < 0 only).  If
                levs2(1) not set by user, levs2 = levs1.
negcon          = 0 => negative contours are indistinguishable       0
                       from positive contours.
                = 1 => negative contours drawn with dotted lines
```

```
hilo            = 0 => highs and lows are not labelled              0
                = 1 => highs and lows are labelled H/L
                = 2 => highs and lows are labelled H/L with values
                       as subscripts (NCAR only).
pl2ccol (niov)  = 0 => do not use colour or grey-scale              0
                = 1 => use grey-scale to fill between contour
                = 2 => use colour     to fill between contour
                if pl2ccol(2:niov) are not set by user, they are
                set to pl2ccol(1).  Used for PSPLOT only.
                pl2ccol < 0 same as abs(pl2ccol) except colour
                bar not drawn (contour levels listed in footer
                instead)
pl2clin1(niov)  controls type of contour lines for first scalar
                = 0 => no     contours (default if pl2ccol .ne. 0)
                = 1 => solid  contours (default if pl2ccol=0 or NCAR)
                =-1 => dashed contours
pl2clin2(niov)  controls type of contour lines for second scalar
                = 0 => no     contours
                = 1 => solid  contours (default if pl2clin1 .ne. 1)
                =-1 => dashed contours (default if pl2clin1=1)
  abs(pl2clin*) = 2 => contour pen weight halved (recommended).

If pl2clin*(2:niov) not set by user, they're set to pl2clin*(1).

Vectors; for vector variables, @ = 1,2,3

pl2vec@ (niov)  names of poloidal vectors to plot (character*2)     'zz'
                Valid names are: 'a ' (vector potential), 'b '
                (magnetic field), 'cf' (centrifugal force), 'gp'
                [grad(p)], 'gr' (gravitational force), 'j '
                (current density), 'lf' (Lorentz force), 's '
                (momentum), 'v ' (velocity), 'w ' (vorticity).
vnorm@  (niov)  >=1.0 => normalise vectors, draw all vectors with   0.0
                         original length > 1.0e-24 * vmax
                > 0.0 => do not normalise vectors, draw vectors
                < 1.0    .ge. vnorm@ * vmax only
                = 0.0 => do not normalise vectors, draw vectors
                         .ge. 0.04  * vmax only
                < 0.0 => normalise vectors, draw all vectors with
                         original length > 10**vnorm@ * vmax
vscale          scaling factor for vector                           0.8
nxvec   (nios)  |nxvec| = # of vectors in x-direction              20
                > 0 => different vector fields are staggered in x
                < 0 => different vector fields have same x-coord
                = 0 => same spacing (not #) as y-direction
nyvec   (nios)  |nyvec| = # of vectors in y-direction               0
                > 0 => different vector fields are staggered in y
                < 0 => different vector fields have same y-coord
                = 0 => same spacing (not #) as x-direction
pl2vcol@(niov)  colours to use for vector fields (PSPLOT only)
                = 0 => white;  = 1 => purple;  = 2 => blue;
                = 3 => green;  = 4 => orange;  = 5 => red;
                = 6 => black
                (Defaults: 6 for 1st field; 5 for 2nd, 2 for 3rd)
                if pl2vcol@(2:niov) are not set by user, they are
                set to pl2vcol@(1).

Corks
```

```
pl2crk  (niov)  vestige from dzeus36; has no effect here.

Diagnostic dumps

dtgr2d          vestige from dzeus36; has no effect here.
```

See note at the end of §1.4.3 concerning libraries required with and without *NCAR* graphics.

```
  namelist / pl2con   /
1               dtpl2   , npl2dmp , units   , pl2scale, pwtpl2
2             , norppl2 , pl2grid , pl2dir  , lpl2    , pl2avg
3             , x1pl2mn , x1pl2mx , x2pl2mn , x2pl2mx , x3pl2mn
4             , x3pl2mx , ipl2mn  , ipl2mx  , jpl2mn  , jpl2mx
5             , kpl2mn  , kpl2mx  , iflippl2, jflippl2, ireflpl2
6             , jreflpl2, npl2i   , npl2j   , pl2xlab , pl2ylab
7             , pl2xl   , pl2yl   , pl2ftr  , pl2leg  , pl2sca1
8             , pl2sca2 , pl2ng1  , pl2ng2  , pl2min1 , pl2min2
9             , pl2max1 , pl2max2 , pl2mm1  , pl2mm2  , thetapl2
1             , phipl2  , numcl1  , numcl2  , levs1   , levs2
2             , negcon  , hilo    , pl2ccol , pl2clin1, pl2clin2
3             , pl2vec1 , pl2vec2 , pl2vec3 , vnorm1  , vnorm2
4             , vnorm3  , vscale  , nxvec   , nyvec   , pl2vcol1
5             , pl2vcol2, pl2vcol3, pl2crk  , dtgr2d
```

2. *RADIO* User Manual

2.1 Introduction, Version 1.3

RADIO is a stand-alone program that encapsulates the line-of-sight integration capability of *ZEUS-3D*, and comes as part of the *ZEUS-3D* (version 3.6) public release tar-file `dzeus36.tar` (but not the abbreviated version `dz36.tar`). Indeed, the first version of *RADIO* was developed before its functionality was imported to *ZEUS*, and derives its name from its original purpose: to compute the Stokes parameters of an astrophysical simulation as though the simulation were being observed by a radio telescope. It has since been extended to include a whole host of variables that can be integrated along the line of sight.

The very first version of *RADIO* used a direct ray tracer to compute the line-of-sight integrations, but this was exceedingly slow and could be as computationally intensive as the early MHD algorithm. Around 1990, I developed a vectorisable algorithm that essentially takes a 2-D slice of the data cube and projects (and rebins) it onto an "image plane" that can be inclined at arbitrary polar angles relative to the axes of the data cube. Each plane so projected then contributes additively to the accumulating signal on the image plane. This algorithm is encapsulated in the subroutine `binint` (`bin`ning and `int`egrating) and sped up the line-of-sight integration process by a factor of >100. It remains the working algorithm in the present version.

RADIO takes one or more "total HDF files" generated by *ZEUS-3D* (format `zhto`*nnn*`id`) as input and, using namelists that—in combination—encompass the *ZEUS-3D* (version 3.6) namelist `radcon`, creates the desired line-of-sight integrations through the datacube from numerous vantage points in the same format as the in-line version of *RADIO* in *ZEUS-3D*. Just as in *ZEUS-3D*, one can also instruct *RADIO* to assemble related sequences of images into mpeg animations, provided the user's platform has the necessary mpeg encapsulation routines available (`FFmpeg` downloadable from `ffmpeg.org`; see discussion in §2.2.3).

The executable for *RADIO* is created in an entirely analogous way as that for *ZEUS-3D*. The program is precompiled, compiled, loaded, and linked by use of the *script file* `radio13.s` (described below), and is managed by the precompiler *EDITOR*. It is assumed the reader has already installed *ZEUS-3D* and *EDITOR* on their unix platform, is familiar with their use, and is looking for a way to visualise data that have been generated by a *ZEUS* run. Therefore, the reader who has not yet installed *ZEUS-3D* on their unix platform should do so first by downloading `dzeus36.tar` from `zeus3d.ca/zeus3d` and following the instructions found therein.

2.2 Running *RADIO*

2.2.1 Installation

This subsection describes how to install *RADIO* on your *UNIX*-based system. For a successful installation, HDF4 libraries must be available. NCAR libraries can be linked if desired, but are not necessary and therefore need not be available on the user's platform.

Having already downloaded `dzeus36.tar` from zeus3d.ca/zeus3d and unpacking the tarball, go to the directory `zeus_v3.6_ICA` and type `ls -FC`. On your screen you should see:

```
README          editor/         manuals/        radio/          zeus/
anim8z/         examples/       nmlst/          sci/
bldlib2*        grfx/           plotz/          testz/
```

In addition to the files in the directory `radio/`, the *EDITOR* executable `editor/xedit22` and the libraries `nmlst/namelist.a`, `grfx/grfx03.a`, `grfx/psplot.a`, and `grfx/noncar.a` (all created during the installation of *ZEUS-3D*) are needed to create the *RADIO* executable, `xradio13`.

Go to the `radio` directory and once again type `ls -FC` where you'll find:

```
README.radio    movies/         radio13         radio13.s
```

The file `radio13` is the source code for *RADIO*, version 1.3 and `radio13.s` is the script file used to compile `radio13` and described below in §2.2.2. The `mp4` files in directory `movies` are for the example in §2.2.3.

As is true for *ZEUS-3D* and *EDITOR*, *RADIO* specifically supports seven compilers: `cf77` (Cray), `f90` (Sun), `g95` (`g95.org`), `gfortran` (Gnu), `ifort` (Intel), `pgf77` (Portland Group), and `xlf` (IBM). If one of these compilers is available on the user's platform, go to step 4.

Step 1. If none of these compilers is available, then suppose the *FORTRAN* compiler that is available is called `newfor`. Open `radio13` in a text editor, replace all occurrences of `GFORTRAN` (note all-caps) with `GFORTRAN.or.NEWFOR`, and save the changes.

Step 2. Open the file `radio13.s` in a text editor, replace the one occurrence of `setenv FCOMP gfortran` with `setenv FCOMP newfor` (note no caps), and save the change.

Step 3. At your unix prompt, type: `csh -v radio13.s`

If this succeeds in creating the executable `xradio13`, installation is complete. If not, it will likely fail by citing calls to unknown implicit functions such as `cpu_time`, `date_and_time`, and/or `system` used by *RADIO* subroutines `RENAM`, `CCLOCK`, and `WCLOCK`. For many compilers such as `gfortran`, these implicit functions do the following:

`cpu_time`	returns the cpu time since the beginning of the run in seconds;
`date_and_time`	returns eight integers giving the year, month, day, *etc.* Search for `date_and_time` in `radio13` for details.
`system`	executes a systems call (such as removing a disc file) at run time.

You will have to find the equivalents to these functionalities for `newfor`. Once determined, open up `radio13` again, find the offending implicit functions, and make the necessary changes.

Step 4. At your unix prompt, type: `csh -v radio13.s`

For those following steps 2 and 3, the complaints about the foreign system calls should disappear. If the executable `xradio13` is created, installation is complete. If not, some

troubleshooting will be necessary for which little general guidance can be given other than the discussion below on libraries (see paragraph beginning with: "Note that in addition to the "exposed"...).

Finally, at the end of the linking step, you may see warning messages of the type:

```
ld: warning: could not create compact unwind for _grfx2d_: stack subq
instruction is too different from dwarf stack size
```

I've no idea what these are, and no one on the "chat groups" seem to be able to explain them to me. What I do know is they seem to be safe to ignore.

2.2.2 The script file `radio13.s`

The script file `radio13.s` used in the previous subsection to install *RADIO* is similar to the script file `dzeus36.s` used to install *ZEUS-3D*. As of this writing, the script file `radio13.s` is as follows:

```
#---+----1----+----2----+----3----+--+----3----+----2----+----1----+---#
#============= SCRIPT FILE TO CREATE THE RADIO EXECUTABLE =============#
#
#----------------------------------------------> Select FORTRAN compiler.
#  Supported compilers include:
#     cf77 (Cray), f90 (Sun), g95 (g95.org), gfortran (Gnu), ifort
#     (Intel), pgf77 (Portland Group), xlf (IBM)
#
setenv FCOMP gfortran
#=========================================> Get files from home directory.
if(! -e ./xedit22) cp ../editor/xedit22 .
if(! -e ./namelist.a) cp ../nmlst/namelist.a .
if(! -e ./grfx03.a) cp ../grfx/grfx03.a .
if(! -e ./psplot.a) cp ../grfx/psplot.a .
if(! -e ./noncar.a) cp ../grfx/noncar.a .
#=======================> If necessary, create the directory "radio1.3".
if(! -e ./radio1.3) mkdir ./radio1.3
#----------------------------------------------> Create the change deck.
rm -f ./chgradio
cat << EOF > chgradio
*define XYZ, MHD, ISO, PLT1D, PLT2D, $FCOMP
*d par.33,34
       parameter     ( in   =  64, jn   =  64, kn   =  64 )
       parameter     ( nxrd = 400, nyrd = 400, nang = 360 )
EOF
#=======================> Create the input deck for EDITOR, and execute.
rm -f ./inedit
cat << EOF > inedit
 \$editpar   inname='radio13', chgdk='chgradio', idump=1, job=3, ipre=1
          , inmlst=1, iupdate=1, iutask=0, ibanner=0, safety=0.50
          , branch='radio1.3', xeq='xradio13', makename='makeradio'
          , compiler='$FCOMP'
c         , coptions='debug'
          , coptions='optimise'
          , libs='namelist.a grfx03.a psplot.a noncar.a
  -L/usr/local/gcc-fortran/lib -ldf -ljpeg -lz -lsz'                  \$
c         , libs='namelist.a grfx03.a psplot.a
c -L/usr/local/gcc-fortran/lib -ldf -ljpeg -lz -lsz -L/opt/local/lib
```

```
 39  c -lncarg -lncarg_gks -lncarg_c -lX11 -lcairo -lfreetype'              \$
 40  c         , libs='namelist.a grfx03.a psplot.a noncar.a'               \$
 41  EOF
 42  chmod 755 ./xedit22
 43  ./xedit22
 44  #-------------------------------------> Create the input deck for RADIO.
 45  rm -f ./inradio
 46  cat << EOF > inradio
 47   \$iocon    intype=1, infile='../plotz/zhto002za', zora=1              \$
 48   \$laingcon                                                            \$
 49   \$initcon  nhy=438, prtime=1.0                                        \$
 50   \$viewcon  nframe=72, theta=30.0, dtheta=5.0, phi=30.0, dphi=5.0
 51           , units='dg', x1radmn=-0.5, x1radmx=0.5, x2radmn=-0.5
 52           , x2radmx=0.5, x3radmn=-0.5, x3radmx=0.5                     \$
 53   \$emitcon  radtype=2, pindex=2.0                                      \$
 54   \$convcon                                                             \$
 55   \$maskcon                                                             \$
 56   \$pl1con   norppl1=0, pl1dir=1,2, lpl1=32,32, pl1mm=0, aspect=1.0
 57           , npl1h=2, npl1v=2, allxl=1, allyl=2, pl1hdr=1, pl1ftr=1
 58           , pl1var='I ','F ','D ','P1','B ','VV','W '
 59           , pl1min= 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0
 60           , pl1max= 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0                  \$
 61   \$pl2con   norppl2=0, pl2mm=0, pl2ftr=1, pl2hdr=1, pl2xl=1, pl2yl=1
 62           , pl2var='IV','FV','D ','P1','B ','VV','W '
 63           , pl2ng =  10,   1,  10,  10,  10,  10,  10
 64           , pl2min= 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0
 65           , pl2max= 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0                  \$
 66   \$veccon   eorb=2, porf=2, bworb=1, vlmin=0.01, icut=5.0e-3
 67           , pcut=2.0e-3, pscale=0.8, incpx= 6, incpy= 6                \$
 68   \$animcon  radfmt=1, radbox=2, fraterad=24
 69           , radvar=   'IV',  'B '
 70           , nlrad =    100,     0
 71           , radmax=6.0e-13,3.0e-5
 72           , radmin=2.5e-15,1.0e-6
 73           , radmm =      2,     1
 74           , radpal=      1,     2                                      \$
 75  EOF
 76  #=========================================> MAKE the RADIO executable.
 77  make -f ./makeradio
 78  #=================================================> Tidy up directory.
 79  rm -f ./editlp ./inedit ./output ./xedit22 ./chgradio ./chgradio.m ./*.a
./radio13.m ./radio13.f
```

As with the *ZEUS-3D* script file `dzeus36.s`, the *RADIO* script file is divided into several segments heralded either by a double line (`====`) or a single line (`----`). Segments heralded by a double line are those which once set should rarely, if ever, be modified. Those heralded with a single line are segments that may have to be modified every time the program is used.

On line 9 of the first segment, the desired *FORTRAN* compiler is selected. Here, `FCOMP` is a "system variable" set by the *UNIX* `setenv` command, and is used in two places. First, on line 21 it serves as the *EDITOR* macro to tailor `radio13` to the chosen compiler. Second, on line 32 it is the setting for the namelist parameter[2] `compiler` in the input deck `inedit` which determines which compiler the makefile uses to compile `radio13`.

Lines 11–15 of the second segment retrieve the files necessary to create the *RADIO* executable and are retrieved only if they do not already exist in the present working directory

[2]See the footnote in the *PLOTZ* manual for a comment on the use of the term *parameter*.

(`pwd`) [`if (! -e` *filename*`)`]. This example assumes that the script file is launched from the directory `radio` created when `dzeus36.tar` from zeus3d.ca/zeus3d is unpacked. Thus, the file `radio13` (containing the source code) is already in this directory, and not retrieved. Files retrieved from other directories include:

`xedit22`	preprocessor executable
`namelist.a`	single precision library of subroutines to enable namelist feature
`grfx03.a`	library of graphics subroutines calling routines in external libraries (*NCAR* and *PSPLOT*)
`psplot.a`	library of routines for *PSPLOT* graphics
`noncar.a`	library of dummy *NCAR* routines used when *NCAR* graphics are not installed at the site

Line 17 of the third segment creates the directory `radio1.3` into which all files split from the master source file `radio13` by *EDITOR* and object files once compiled are placed.

The fourth segment (lines 19–25) generates the change deck `chgradio` which gets merged with the master file `radio13` by *EDITOR* during the precompilation step. Line 21 sets the various *EDITOR* macros necessary to tailor the compilation of *RADIO* (corresponding with the *EDITOR* macros used to compile *ZEUS-3D* that created the total HDF files *RADIO* will be expected to read), along with the compiler used to compile the program. Lines 22–24 set the array dimensions for the *ZEUS*-like variables stored in the HDF files as well as the desired dimensions of the *RADIO* image plane and the maximum number of vantage points. See §2.2.4 for further discussion.

The fifth segment (lines 27–43) generates the namelist input data file `inedit` which the *EDITOR* executable `xedit22` uses to precompile `radio13`. In this example, the namelist parameters are set so that `chgradio` will first be merged into `radio13` which is then precompiled (since `ipre=1`) for `gfortran` with the namelist feature *on* (`inmlst=1`), the conditional splitting feature *on* (`iupdate=1`), and the micro-tasking feature *off* (`iutask=0`). All files split from the merged master source code will be placed in the directory `radio1.3` with the extension `.f`, appropriate for *FORTRAN* source code. Since `iupdate=1` *and* `makename` is set to a non-blank character string in the input deck, a makefile with the name `makeradio` will be generated. The makefile will use the compiler `gfortran -c` and loader `gfortran` with the highest safe optimisation (`coptions='optimise'` not commented out). The specified libraries will be linked with the *RADIO* object code to generate a binary executable called `xradio13`.

Note that in addition to the "exposed" list of libraries (lines 35–36) which assumes HDF4 libraries are available but not NCAR, the commented out list of libraries (lines 37–39) can be used if the NCAR libraries have been installed on the user's platform. As given, these lists are tailored for my own platform and will almost certainly fail on the user's platform. Indeed, it is with these lists that the new user may have to spend much time troubleshooting. Linking third-party libraries can be frustratingly tiresome, particularly as various installations and versions will have different library requirements, locations, *etc.*, and the user may well have to consult liberally with their systems administrator to get the right libraries in place and to make sure they are listed correctly in `radio13.s`. The last library list (line 40) is one stripped of HDF4 and NCAR and thus should link without errors. However, without HDF4,

RADIO is rather emasculated, as it will not be able to read any HDF dump files generated by *ZEUS-3D*.

Finally in this segment, line 43 launches `xedit22` and `radio13` is precompiled.

The sixth segment (lines 45–75) sets the namelist input deck, `inradio`, used by *RADIO* itself to generate the desired output from the specified input total HDF file(s). All namelist parameters for *RADIO* are described in §2.4.

Line 77 in the seventh segment fires up the makefile `makeradio` generated in the fifth segment. The result of this step is the executable, `xradio13`.

Finally, line 79 of the last segment removes the intermediate files no longer needed.

2.2.3 A quick example using *RADIO*

If `radio13.s` executed successfully in the previous subsection, there should be an input deck `inradio` (lines 47–74 in `radio13.s` described above, without the backslashes before the $ signs) and an executable `xradio13` in your `pwd`. These two files plus the data file `../plotz/zhto002za` are all that are needed to run the example provided herein.

So, remaining within the `radio` directory, type `xradio13` at the unix prompt, and *RADIO* will fire up, making the plots and/or animations as requested by the input deck `inradio` from the total HDF dump `zhto002za` which can remain in the directory `plotz` by virtue of the `../plotz/` in the file specification (parameter `infile` in namelist `iocon`). This HDF file was taken from a very low-resolution (64^3) MHD simulation of a super-Alfvénic turbulent medium, one thought to model portions of the interstellar medium.

As given in `inradio`, 1-D and 2-D plots are specified in namelists `pl1con` and `pl2con` but suppressed by virtue of `norppl1=norppl2=0` (since they would generate a large number of files). In order to generate the 1-D (2-D) plots, `norppl1` (`norppl2`) must be set to `1` for *NCAR*'s metacode format, or `2` for *PSPLOT*'s postscript format.

Thus, as given, only four files are produced, two for each movie requested in namelist `animcon`:

`rRB_002za.mp4, rRB_002za.tar.gz`	magnetic pressure
`rRIV002za.mp4, rRIV002za.tar.gz`	total intensity plus polarisation vectors

The `.mp4` files are the two animations and the compressed tar files ("tarballs"; `.tar.gz`) contain all the `real*4` movie frames in "movie" format (see §2.3 for a discussion of the types of files generated by *RADIO*) that may be used to reprocess the animation should the user wish to change any of the settings such as the bracketing o logarithmic compression without running *RADIO* again (using, for example, *ANIM8Z*; see §2.4.10).

To create the mpegs, the user's platform must support `FFmpeg`, a commonly available mpeg encoder which the user can download from `ffmpeg.org` if desired. Without `FFmpeg`, *RADIO* issues a warning message that the mpeg files cannot be written, though the tarballs—relying only on subroutines internal to *RADIO*—will still be created. These can be ported to a different platform, for example, that supports `FFmpeg` and on which *ANIM8Z* could be used to create the mpegs (§2.4.10).

The user is encouraged to view the animations `rRB_002za.mp4` and `rRIV002za.mp4` in their favourite mpeg viewer. In case the user's platform does not support `FFmpeg`, these files

as generated on my laptop are included in the directory `movies`.

By virtue of the namelist parameter settings in `viewcon`, the animations consist of 72 frames, each at a different vantage point which, when strung in succession, give the impression of the data cube revolving once about an axis. By virtue of the settings in the namelist `animcon`, the two animations include the magnetic pressure integrated along the line-of-sight through the data cube (`radvar='B '`), and the synchrotron total intensity with fractional polarisation B-field vectors superposed (`radvar='IV'`). (By the way, it is physically significant that the magnetic pressure traces the synchrotron emission.) A box, with the corner furthest from the observer hidden from view, is drawn around the data cube (`radbox=2`) to guide the eye of the observer.

This example has touched on only a few of the namelist parameters available to control how *RADIO* functions. Note that most namelist parameters have sensible defaults, and thus need not be mentioned in the input deck `inradio` to have an effect on the appearance of the plots and animations. All namelist parameters for *RADIO* are described in detail in §2.4.

2.2.4 *RADIO* parameters

There are only a few parameters of which one must be aware when running *RADIO*, all but one familiar to *ZEUS-3D* users and all listed in lines 23–24 in the script file `radio13.s` in §2.2.2. These are:

parameter	interpretation	default
in	number of active zones in 1-direction in ZEUS-3D run	
jn	number of active zones in 2-direction in ZEUS-3D run	
kn	number of active zones in 3-direction in ZEUS-3D run	
nxrd	maximum number of pixels in x-direction of image plane	
nyrd	maximum number of pixels in y-direction of image plane	
nang	maximum number of vantage points for los integrations	360

Note that `in`, `jn`, and `kn` should all be set to *five* less than they were in the *ZEUS-3D* simulation that generated the HDF dumps being read by *RADIO*. This is because HDF dumps do not store the ghost zones, only the active ones. Parameters `nxrd` and `nyrd` need not be the same as they were in *ZEUS-3D* but can be, while parameter `nang` has no *ZEUS-3D* analogue.

2.3 Output from *RADIO*

In addition to a log-file, *RADIO* generates four types of output files. Two are 1-D and 2-D dumps in either *NCAR*'s metacode or postscript generated by *PSPLOT* and are of publication quality. These are plots through and of the image planes, and have no analogue in *ZEUS-3D* which performs 1-D and 2-D plots through slices of the datacube itself. The other two files are mpegs and movie tar files, identical in form to those generated by *ZEUS-3D* at run time.

This section describes the characteristics of the output files including the namelist which controls the data dump (§2.4), the convention used for naming the file, and the format of the data in the file created.

2.3.1 1-D plot files

These are 1-D slices through the image plane parallel to either the horizontal or vertical axes of the requested variables integrated along the line of sight at the vantage point selected. Data are plotted continuously with smooth lines. A separate plot file containing slices of each of the variables requested is created for each slice and each vantage point specified.

namelist:	`pl1con`
filename:	`rp`*nnn*`id.`*mm*`.`*vvv*, where `rp` is the common prefix to all 1-D plot files generated by *RADIO*, *nnn* is the three-digit sequential number parsed from the input HDF file, `id` is the two-character ID tag also parsed from the HDF file, *mm* is a two-digit extension indicating the slice number, and *vvv* is a three-digit extension enumerating the vantage point. For *PSPLOT*, the suffix `.ps` is added to the filename.
format:	metacode—use `idt` to read *NCAR*-generated metafiles postscript—use `mgv`/`gv` to read *PSPLOT*-generated postscript files

The format of these files is similar to the *ZEUS*-created files `zp`*nnn*`id.`*mm*(`.ps`), though not the content.

2.3.2 2-D plot files

2-D colour-contour plots of any or all the line-of-sight variables may be created, including Stokes variables (with or without polarisation vectors superposed) and the various flow variables designated for line-of-sight integrations (see §2.4.10 for a list of the allowed variables). Unlike *ZEUS-3D* and *PLOTZ* which use the "buffet-style" plotting, the 2-D plots available in *RADIO* are all preset—one cannot, for example, superpose contours of the magnetic pressure line-of-sight integration over the total intensity image.

Since the image plane is already a 2-D data set, there are no 2-D slices per se; each integration yields just one 2-D plane for plotting, and thus there is no slice extension (*e.g.*, *mm*) on the 2-D plot filenames. Note, however, that a separate plot file is generated for every vantage point specified.

namelist:	`pl2con`
filename:	`rq`*nnn*`id.`*vvv*, where `rq` is the common prefix to all 2-D plot files generated by *RADIO*, *nnn*, `id`, and *vvv* are as described for 1-D plots. For *PSPLOT*, the suffix `.ps` is added to the filename.
format:	metacode—use `idt` to read *NCAR*-generated metafiles postscript—use `mgv`/`gv` to read *PSPLOT*-generated postscript files

The format of these files is similar to the *ZEUS*-created files `zq`*nnn*`id.`*mm*(`.ps`), though not the content.

2.3.3 Movie frames

For each input HDF file and for each vantage point selected, a separate image for every selected variable may be created for use in generating animations. The number of movie

frames thus generated can therefore become enormous! With the right format selected, however, *RADIO* will reduce these files to just two per movie—a tar file, and an mpeg. This subsection describes the intermediate files which make up the individual frames of these animations.

Movie frames may be written in one, two, or all three of three formats: `real*4` "movie" (`radfmt=1`); Mike Norman's `char*1` "pixel dumps" (`radfmt=2`); and HDF4 (`radfmt=3`), with the latter suitable for graphical analysis if desired. Files in movie format are "managed" by *RADIO* in the same way *ZEUS-3D* manages them. At the end of the job, all files with extension `.m` are converted to `PPM` format (by virtue of routines internal to *RADIO*) and strung together to make an mpeg (using `FFmpeg`, as discussed in §2.2.3). The `PPM` files are then deleted (provided `ppmrad=0`) and the `.m` files are compressed into a single tar ball before they are deleted (provided `tarrad=1`). Thus, for `radfmt=1` only two movie-related files are produced for every variable of every HDF input file.

If either pixel or HDF files are written, none of these files are managed and there will be a number of files equal to the number of input HDF files times the number of variables times the number of pixel files per animation to manage manually.

namelist:	`animcon`
filename:	`rR**`*nnn*`id.`*vvv*{`.f`}, where `rR` is the common prefix to all movie frames generated by *RADIO*, `**` is a two-character variable designation, *nnn*, `id`, *vvv* are as defined for 1-D plots (§2.3.1), and where `f` is one of `m` for movie format (`radfmt=1`), blank for pixel dumps (`radfmt=2`), or `h` for HDF4 (`radfmt=3`). On exit, the `.m` files are assembled in a tarball `rR**`*nnn*`id.tar.gz`, then converted to `PPM` files which are used to create the mpeg `rR**`*nnn*`id.mpg/mp4`.
format:	`radfmt=1`: unformatted, four bytes per datum plus header `radfmt=2`: unformatted, one byte per datum, no header `radfmt=3`: HDF4

2.3.4 Message log files

The message log file contains all the messages that are written to the terminal during execution, and serves as a record of the run. This feature is inherited from *ZEUS-3D*, where such files are more useful. For *RADIO*, these files are largely vestigial.

namelist:	`none`
filename:	`rl`*nnn*`id`, where `rl` is the common prefix to all log files generated by *RADIO*, *nnn* and `id` are as defined for 1-D plot files.
format:	ascii

2.4 Namelists for *RADIO*

Most of the `namelist` parameters used by *RADIO* are identical to those in the namelist `radcon` in *ZEUS-3D*, although here, they are divided up among various smaller-tasked namelists. In addition, namelists `pl1con` and `pl2con` control the 1-D and 2-D plots of quantities integrated along the line of sight, and are not particularly related to the same-named namelists in `dzeus36` which control 1-D and 2-D plots along slices through the data

cube.

This section gives a complete catalogue of all the namelist parameters in `radio13`. As with the *ZEUS* manual, all parameters are grouped together in separate namelists and discussion for each namelist is contained within a subsection headed by the name of the namelist and the `radio13` subroutine in which the namelist is called. After each heading is a discussion of what the namelist controls, a list of all the parameters which are elements of the namelist, and finally the syntax used in `radio13` to declare the namelist. For a reminder of the nature of namelists and how they are used in the *ZEUS* family of codes, the reader is referred to the first page of Appendix B in the `dzeus36` user manual.

2.4.1 `IOCON`—`I/O CON`trol (subroutine `IOINIT`)

RADIO can use *ZEUS-3D* HDF total files as input (`intype=1`), or generate its own data assigning field variables appropriate for Robert Laing's helical jets (1981, ApJ, 248, 87; `intype=2`), or cylinders with constant flow variables (`intype=3`). The latter two are included for testing purposes.

For `intype=1`, this namelist controls which total HDF files are read by `radio13` during execution. One can specify a single file (`infile`) or, if multiple files are to be read and have the same line-of-sight integrations performed on them, one can further specify `lastfile` in the sequence to be read (`infile` being the first). Here, the usual format for *ZEUS* HDF files (`zhto`*nnn*`id`) is assumed, and both `infile` and `lastfile` must have the same two-character `id`. The routine starts with `infile`, adds `mdinc` to *nnn* and reads that file, *etc.* until `lastfile` is read. If any file along the way is not found, execution is aborted.

```
parameter                     description                        default

intype          =1 => read a ZEUS-3D/AZEuS HDF file (default)
                =2 => fill ZEUS variables with a Laing-type jet
                =3 => fill ZEUS variables with constant data
infile          character*32 name of first HDF file to be read
lastfile        character*32 name of last  HDF file to be read
                The 32 characters allow for a modest directory
                path to precede filename, if desired.
zora            =1 => infile and lastfile are ZEUS-3D files of       1
                      form: zhtoNNNID, where "NNN" is a 3-digit
                      sequential number and "ID" is a two-
                      character problem tag
                =2 => infile and lastfile are AZEuS files of
                      form: ahtoNNNID.GG, where "NNN" and "ID are
                      as above, "GG" is two-digit grid identifier
mdinc           increment between files.  0 => read only "infile"    0
                e.g., if every 3rd file zhto001aa --> zhto100aa
                is to be read, set mdinc=3.  If every file is to
                be read, set mdinc=1.

       namelist / iocon    /
     1               intype  , infile  , lastfile, zora    , mdinc
```

2.4.2 INITCON—INITialisation CONtrol (subroutine NMLSTS)

This namelist includes the relevant *ZEUS*-parameters which help characterise the simulation that were not stored with the HDF file. These will be needed, for example, if the pressure has to be calculated from the internal energy (requiring the adiabatic index), or if the fluid is isothermal (and the isothermal sound speed is needed). Further, 1-D and 2-D plots are labelled with the problem time (`prtime`) and the number of cycles of execution (`nhy`) corresponding to when the HDF file was created, and these too have to be input.

```
parameter                     description                          default

gamma           adiabatic index of two fluids                        5/3
ciso            isothermal sound speed of two fluids                 1.0
prtime          problem time of input (A)ZEUS HDF file
nhy             MHD cycle of input (A)ZEUS HDF file
                "prtime" and "nhy" are needed only to label graphs

       namelist / initcon  /
     1               gamma   , ciso    , prtime  , nhy
```

2.4.3 VIEWCON—VIEWing CONtrol (subroutine NMLSTS)

This namelist controls the vantage point of the viewer and the region of the simulation through which line-of-sight integrations should be performed.

The vantage point is specified by two inclination angles, ϕ and θ. To understand how these angles are defined, orient the data cube so that the x_1 and x_2 axes are aligned with the x (horizontal) and y (vertical) axes of the image plane. For 3-D Cartesian coordinates, the x_3 axis then points toward the observer, assumed to be at infinity. Then, ϕ is the angle of rotation about the x_1 axis so that the $+x_2$ axis tilts towards the observer, and θ is the angle of rotation *about the original* $-x_2$ *axis* so that the $+x_1$ axis tilts toward the observer. Note that the rotation ϕ is applied before the rotation θ. For 2-D cylindrical coordinates, the rotation ϕ is moot since it happens about the symmetry axis.

To specify angles ϕ (`phi`) and θ (`theta`), each a 1-D array with dimension `nang`, one can *either*:

1. For non-uniformly spaced vantage points, set `nframe` to 0 and specify as many as `nang` values of `phi` and/or `theta` directly (*e.g.*, `phi=phi1,phi2,phi3,...`); *or*

2. For uniformly spaced vantage points, set `nframe` to the number of vantage points required, `phi(1)=phimin`, `theta(1)=thetamin`, and then specify `dphi` and `dtheta` (increments between viewing angles). Setting `dphi` and/or `dtheta` overrides all values in `phi` and `theta` except `phi(1)` and `theta(1)` which are used to set the initial vantage point.

```
parameter                     description                          default

units           sets units of angular dimensions (character*2).      'rd'
                'dg' => degrees, 'rd' => radians, 'pi' => pi
                radians
nframe       .le. 0  => use viewing angles in "theta" and "phi".     0
```

```
                > 0  => interpret theta(1)=thetamin, phi(1)=phimin
                        Increment "theta" and "phi" by "dtheta"
                        and "dphi" between successive frames, for
                        a maximum of "nframe" frames.
theta   (nang)  inclination of x1-axis toward observer.            0.0
dtheta          increment for "theta" between integrations.        0.0
phi     (nang)  inclination of x2-axis toward observer (3-D only)  0.0
dphi            increment for "phi" between integrations.          0.0
x1radmn         minimum x1 for RADIO integration                   x1a(is)
x1radmx         maximum x1 for RADIO integration                   x1a(iep1)
x2radmn         minimum x2 for RADIO integration                   x2a(js)
x2radmx         maximum x2 for RADIO integration                   x2a(jep1)
x3radmn         minimum x3 for RADIO integration                   x3a(ks)
x3radmx         maximum x3 for RADIO integration                   x3a(kep1)
radfill         = 0 => diameter of image plane set to:             0
                       sqrt (x1radln**2 + x2radln**2 + x3radln**2)
                       so that no matter the viewing angle, image
                       fits entirely on image plane.
                = 1 => diameter of image plane set to x1radln so
                       that viewed face-on, image fills entire
                       image plane, leaving no black around image.

      namelist / viewcon  /
     1              units   , theta   , dtheta  , phi     , dphi
     2            , nframe  , x1radmn , x1radmx , x2radmn , x2radmx
     3            , x3radmn , x3radmx , radfill
```

2.4.4 EMITCON—EMIssion CONtrol (subroutine NMLSTS)

This namelist contains the parameters which control the type of emission desired (synchrotron and bremsstrahlung). Parameters `radtype` through `betarel` are identical to those found in the namelist `radcon` in *ZEUS-3D*, while those following are for subtracting a King model from the Bremsstrahlung emissivity, a feature not supported in *ZEUS*.

```
parameter                    description                        default

radtype         = 0 => emissivities are not computed.              2
                = 1 => Smith et al emissivity (p**2)
                = 2 => sync. emissivity (function of d, p, B)
                = 3 => sync. emissivity, including break freq.
                = 4 => unity (for testing)
pindex          power law index of relativistic electrons          2.0
                N(E) = kappa * E**(-pindex) (radtype=2,3)
freq            frequency of RADIO observation (Hz)                5.0e9
nubrs           nubr(Hz)/nubr(zeus); scaling factor for break      0.5
                frequency (radtype=3)
brn0            fiducial number density for bremsstrahlung         lrge
                emission (m**-3)
brt0            fiducial temperature for bremsstrahlung            lrge
                emission (K)
brnu1,brnu2     limits of frequency band (Hz)                      1.0000e17,
                                                                   1.0001e17
betarel         relativistic beta = v/c for fastest zone (for      0.0
                Mike Casey's Doppler boosting)
x10k,x20k,x30k  coordinates at centre of King atmosphere.        grid centre
r0k             core radius                                        1.0
betak           value of exponent (betak = <v>/kT)                 0.75
```

```
d0k             central density                                        1.0
e0k             energy density (constant for isobaric atmosphere)      0.6
eod0k           specific energy density (constant for isothermal       0.9
                atmosphere)
portk           =1.0 => isobaric, =2.0 => isothermal.                  1.0

       namelist / emitcon  /
     1               radtype , pindex  , freq    , brn0    , brt0
     2             , brnu1   , brnu2   , betarel , x10k    , x20k
     3             , x30k    , r0k     , betak   , d0k     , e0k
     4             , eod0k   , portk
```

2.4.5 CONVCON—CONVolution CONtrol (subroutine NMLSTS)

This namelist includes the parameters to set the "convolution beam" used exclusively for the Stokes parameters and the "Q-SRA variables". The parameter `cpb` is an accuracy parameter whose default (5.0) should be fine in most cases. Setting it lower will speed up the computation, but will yield a more ragged image.

All parameters are identical to those found in the namelist `radcon` in *ZEUS-3D*.

```
parameter                    description                          default

cnvlv           = 0 => do not apply convolution                       0
                = 1 => apply convolution to Stokes parameters.
bmajor          major axis of convolving beam in distance units       1.0
bminor          minor axis of convolving beam in distance units       1.0
bpa             beam position angle (radians) measured counter-       0.0
                clockwise from major axis to + xtw axis
cpb             "cells" per beam                                      5.0

       namelist / convcon  /
     1               cnvlv   , bmajor  , bminor  , bpa     , cpb
```

2.4.6 MASKCON—MASKing CONtrol (subroutine NMLSTS)

This namelist contains the parameters to control how the line-of-sight integrations of the non-Stokes variables are "masked", and thus limited by properties of the integrand or on properties of the density (`d`) and/or the magnetic field strength (`b`). For example, if a contact discontinuity (CD) encloses the region of interest, then there will be a density jump along this interface. Thus, setting `dmask*=1.0` and `dupper=0.5` (if, for example, ρ jumps from about 0.1 to about 1.0 across the CD) would allow only the low density region (interior or exterior to the CD) to contribute to the line-of-sight integration of variable `*` (see list of allowed variables below). Alternatively, if the material of interest is magnetised, setting `bmask*=1.0` and `blower=0.001` (something small) would allow variable `*` to be included in the line-of-sight integration only in zones where $B > 0.001$. Finally, the variables `*lower` and `*upper` allow each variable to be masked by its own distribution. These can be set in addition to the density and/or magnetic field masks. For example, if only the compressive velocity divergences are to be integrated, then setting `vvupper=0.0` will mean that only negative values of $\nabla \cdot \vec{v}$ (*e.g.*, shocks) will be included in the integration. All values excluded by the various masks are given zero weight. In all cases, the default is no mask.

In what follows:

```
* = d, p1, t , b, sh, vv, br, w, m, ma, ms, mf, db, eb, de
q = d, e, e/d, B, shear, divv, brem, curlv, M, MA, MS, MF, dB, Be**2, d/e**2
```

All parameters are identical to those found in the namelist `radcon` in *ZEUS-3D*.

```
parameter                    description                        default

*lower          q is integrated along los provided q > *lower.      -huge
*upper          q is integrated along los provided q < *upper.       huge
dmask*          density mask toggle for variable * (except "d")     -1.0
                = 1.0 => "dlower", "dupper" set integration limits
                = 0.0 => not
                =-1.0 => use "dmask"
dmask           density mask toggle for all variables.  If           0.0
                "dmask*" .ne. -1.0, value of "dmask*" overrides
                "dmask" for variable *.
bmask*          B-field mask toggle for variable * (except "b")     -1.0
                = 1.0 => B-field extent sets integration limits
                = 0.0 => not
                =-1.0 => use "bmask"
bmask           B-field mask toggle for all variables.  If           0.0
                "bmask*" .ne. -1.0, value of "bmask*" overrides
                "bmask" for variable *.

       namelist / maskcon  /
     1               dlower  , p1lower , tlower  , blower  , shlower
     2             , vvlower , brlower , wlower  , mlower  , malower
     3             , mslower , mflower , dblower , eblower , delower
     4             , dupper  , p1upper , tupper  , bupper  , shupper
     5             , vvupper , brupper , wupper  , mupper  , maupper
     6             , msupper , mfupper , dbupper , ebupper , deupper
     7             , dmaskp1 , dmaskt  , dmaskb  , dmasksh , dmaskvv
     8             , dmaskbr , dmaskw  , dmaskm  , dmaskma , dmaskms
     9             , dmaskmf , dmaskdb , dmaskeb , dmaskde , dmask
     1             , bmaskd  , bmaskp1 , bmaskt  , bmasksh , bmaskvv
     2             , bmaskbr , bmaskw  , bmaskm  , bmaskma , bmaskms
     3             , bmaskmf , bmaskdb , bmaskeb , bmaskde , bmask
```

2.4.7 PL1CON—PLot (1-D) CONtrol (subroutine NMLSTS)

This namelist includes input parameters for 1-D line plots. While the names and functionality of many of the parameters are the same as those in namelist `pl1con` in `dzeus36` and `plotz13`, the nature of the plots produced here is different. In *ZEUS* and *PLOTZ*, 1-D plots are taken along 1-D slices directly through the 3-D data cube. In *RADIO*, 1-D plots are are taken either along the $\tilde{x}$- or $\tilde{y}$-axis of the image plane which contains the line-of-sight integrations of selected variables at selected vantage points. Such plots are possible only in *RADIO*, as *ZEUS-3D* does not have the capacity to dump line-of-sight integrations in any format other than movie frames.

File names for 1-D plots have the form `rp`*nnn*`id.`*vvv*`.`*mm*, where each segment is defined in §2.3.1. For postscript files (`norppl1=2`), there is an additional extension `.ps`. Note that one 1-D plot file with slices of all variables requested is generated per input HDF file per vantage point, and per slice requested. Thus, numerous 1-D plot files could result if not selected judiciously.

```
parameter                    description                        default

norppl1          = 1 => use NCAR   graphics for 1-D plots          2
                 = 2 => use PSPLOT graphics for 1-D plots
                 = 0 => no plots are generated
pl1var  (niov)   character*2 mnemonics for variables to be plotted
                 using 1-D graphics.  Legal mnemonics are:
                 'I ' => total intensity (stki)
                 'P ' => polarised intensity (poli)
                 'F ' => fractional polarisation (fpol)
                 'A ' => polarisation position angle (pola)
                 'D ' => column density
                 'P1' => column pressure (int. energy density)
                 'B ' => column magnetic field strength
                 'T ' => column temperature (specific int. energy)
                 'SH' => column velocity shear
                 'VV' => column divergence of velocity
                 'BR' => bremsstrahlung emission
                 'W ' => vorticity
                 'M ' => Sonic Mach number
                 'MA' => Alfven Mach number
                 'MS' => Fast magnetosonic Mach number
                 'MF' => Fast magnetosonic Mach number
                 'DB' => d * B     (K-S R A variable)
                 'EB' => p1**2 * B (K-S R A variable)
                 'DE' => d / p1**2 (K-S R A variable)
nlpl1   (niov)   = 1 => linear-linear    plot                      1
                 = 2 => log(y)-linear(x) plot
                 = 3 => linear(y)-log(x) plot
                 = 4 => log-log          plot
pl1min  (niov)   minimum value to be plotted.                      0.0
pl1max  (niov)   maximum value to be plotted.                      0.0
pl1ref  (niov)   reference line to be drawn on plot                no line
pl1mm            = 0 => use input "pl1min", "pl1max" for each plot  1
                 = 1 => compute "pl1min", "pl1max" for each plot.
                        Setting "pl1min" and "pl1max" to 0.0 for a
                        particular variable is equivalent to
                        setting "pl1mm" to 1 for that variable.
pl1dir  (nios)   = 0 => no plots generated                         0
                 = 1 => xtw-direction of image plane
                 = 2 => ytw-direction of image plane
lpl1    (nios)   index of slice                                    1
xtwpl1mn(nios)   minimum xtw of slice on image screen            entire screen
xtwpl1mx(nios)   maximum xtw of slice on image screen
ytwpl1mn nios)   minimum ytw of slice on image screen            entire screen
ytwpl1mx(nios)   maximum ytw of slice on image screen
ilaing           = 1 => use Laing's sign convention for fractional  1
                        polarisation
                 = 0 => Laing's sign convention not used
inorm            = 1 => normalise plots                            1
                 = 0 => do not normalise plots
aspect           < 1 => plots are wide and short                   1.0
                 = 1 => square plots
                 > 1 => plots are tall and narrow
npl1h            number of plots horizontally per frame            1
npl1v            number of plots vertically   per frame            1
allxl            = 2 => all x labels/annotations are drawn         2
                 = 1 => only x-label/annotations on bottom row of
                        plots on frame are drawn.
```

```
                = 0 => labels aren't even drawn on bottom row.
allyl           = 2 => all y labels/annotations are drawn            2
                = 1 => only y-label/annotations on left column of
                       plots on frame are drawn.
                = 0 => labels aren't even drawn on left column.
pl1hdr          = 0 => suppresses header                             1
                = 1 => header is printed on top    of plot
pl1ftr          = 0 => suppresses footer                             1
                = 1 => footer is printed on bottom of plot

"ilaing" and "inorm" affect plots of the Stokes parameters only.

       namelist / pl1con  /
     1              norppl1 , pl1var  , nlpl1   , pl1min  , pl1max
     2            , pl1ref  , pl1mm   , pl1dir  , lpl1    , xtwpl1mn
     3            , xtwpl1mx, ytwpl1mn, ytwpl1mx, ilaing  , inorm
     4            , aspect  , npl1h   , npl1v   , allxl   , allyl
     5            , pl1hdr  , pl1ftr
```

2.4.8 PL2CON—PLot (2-D) CONtrol (subroutine NMLSTS)

This namelist includes input parameters for 2-D (colour) contour plots. Parameters for polarisation vector overlays (which also apply to movie frames) are controlled by namelist `veccon` (§2.4.9). While the names and functionality of many of the parameters are the same as those in namelist `pl2con` in `dzeus36` and `plotz13`, the nature of the plots produced here is different. In *ZEUS* and *PLOTZ*, 2-D plots are taken along 2-D slices directly through the 3-D data cube. In *RADIO*, 2-D plots are of line-of-sight integrated quantities and cover the entire image plane. It therefore makes no sense to specify a slice. These 2-D plots are unique to *RADIO*; *ZEUS* itself does not support visualising the image plane in this way.

File names for 2-D plots have the form `rq`*nnn*`id.`*vvv*, where each segment is defined in §2.3.2. For postscript files (`norppl2=2`), there is an additional extension `.ps`. Note that one 2-D plot file with images of all variables requested is generated per input HDF file per vantage point.

```
parameter                     description                        default

norppl2         = 1 => use NCAR   graphics for 2-D plots             2
                = 2 => use PSPLOT graphics for 2-D plots
                = 0 => no plots are generated
pl2var  (niov)  character*2 mnemonics for variables to be plotted
                using 2-D graphics.  Legal mnemonics are:
                'V ' => polarisation vectors only
                'I ' => total intensity (toti) contours only
                'IV' => toti contours with polarisation vectors
                'P ' => polarised intensity (poli) contours only
                'PV' => poli contours with polarisation vectors
                'F ' => fractional polarisation (fpol) contours only
                'FV' => fpol contours with polarisation vectors
                'A ' => pol. position angle [ 0.5 * atan (Q/U) ]
                'AV' => A with polarisation vectors superposed
                'D ' => column density
                'P1' => column pressure (internal energy density)
                'B ' => column magnetic field strength
                'T ' => column temperature (specific internal energy)
```

```
                 'SH' => column velocity shear
                 'VV' => column divergence of velocity
                 'BR' => bremsstrahlung emission
                 'W ' => vorticity
                 'M ' => Sonic Mach number
                 'MA' => Alfven Mach number
                 'MS' => Fast magnetosonic Mach number
                 'MF' => Fast magnetosonic Mach number
                 'DB' => d * B     (K-S R A variable)
                 'EB' => p1**2 * B (K-S R A variable)
                 'DE' => d / p1**2 (K-S R A variable)
pl2ng   (niov)   = 1 => contours are spaced evenly                    1
                 > 1 => contours are spaced geometrically such
                        that first two contours are spaced "ngeom"
                        times closer than evenly spaced contours.
                 <-1 => contours are spaced geometrically such
                        that last two contours are spaced "-ngeom"
                        times closer than evenly spaced contours.
pl2max  (niov)   vector specifying maximum contour in each plot       0.0
pl2min  (niov)   vector specifying minimum contour in each plot       0.0
pl2mm            = 0 => use input "pl2min", "pl2max" for each image   1
                 = 1 => compute "pl2min", "pl2max" for each image
                 Setting "pl2min", "pl2max" to 0 for a particular
                 variable is equivalent to setting "pl2mm" to 1 for
                 that variable.
numcl            > 0 => number of evenly spaced contours to draw      ncls
                        between "pl2min" and "pl2max" (max ncls).
                 < 0 => abs(numcl) contours drawn and are specified
                        by "pl2min" and "levs".
levs    (ncls)   real array specifying multiples of "pl2min" to use   0.0
                 for contour levels (for numcl < 0 only).
pl2ccol (niov)   = 0 => do not use colour or grey-scale               2
                 = 1 => use grey-scale between contours
                 = 2 => use colour     between contours
                 if pl2ccol(2:niov) are not set by user, they are
                 set to pl2ccol(1).  Used for PSPLOT only.
                 pl2ccol < 0 same as abs(pl2ccol) except colour bar
                 not drawn (contour levels listed in footer instead)
pl2clin (niov)   controls type of contour lines
                 = 0 => no     contours (default if pl2ccol .ne. 0)
                 = 1 => solid  contours (default if pl2ccol=0 or NCAR)
                 = 2 => dashed contours
pl2ftr           = 0 => suppresses footer                             1
                 = 1 => footer is printed on bottom of plot
pl2hdr           = 0 => suppresses header                             1
                 = 1 => header is printed on top    of plot
pl2xl            = 0 => suppresses xlabel (but not x-annotations)     1
                 = 1 => xlabel is put below horizontal axis
pl2yl            = 0 => suppresses ylabel (but not y-annotations)     1
                 = 1 => ylabel is put beside vertical  axis

      namelist / pl2con  /
     1              norppl2 , pl2var  , pl2ng   , pl2max  , pl2min
     2            , pl2mm   , numcl   , levs    , pl2ccol , pl2clin
     3            , pl2ftr  , pl2hdr  , pl2xl   , pl2yl
```

2.4.9 VECCON—VECtor CONtrol (subroutine NMLSTS)

This namelist includes parameters for controlling how vectors are laid over both the NCAR/PSPLOT 2-D plots (`pl2con`, §2.4.8) and the movie frames (`animcon`, §2.4.10). Parameter `bworb` applies only to the movie frames; all other parameters apply to both.

All parameters are identical to those found in the namelist `radcon` in *ZEUS-3D*.

```
parameter                    description                        default

eorb            = 1 => E-vectors                                   2
                = 2 => B-vectors
porf            = 1 => vector length proportional to poli          2
                = 2 => vector length proportional to fpol
bworb           = 1 => black and white pixel vectors               1
                = 2 => black pixel vectors only
vlmin           vectors with length < vlmin*(max vector) not       0.001
                plotted
icut            vectors are not plotted if toti < icut*max(toti)   0.001
pcut            vectors are not plotted if poli < pcut*max(poli)   0.001
polsc           scaling factor for polarisation vectors            0.8
                = 1 => vectors lining up along image axis touch
                < 0 => vectors scaled by |polsc| and have
                       uniform length (useful to map polA)
incpx           plot every incpx(th) vector in xtw-direction       1
incpy           plot every incpy(th) vector in ytw-direction       1

       namelist / veccon  /
     1               eorb    , porf    , bworb   , vlmin   , icut
     2             , pcut    , polsc   , incpx   , incpy
```

2.4.10 ANIMCON—ANIMation CONtrol (subroutine NMLSTS)

This namelist includes the input parameters that specifically control what movie frames are generated, and how they are produced. Parameters for polarisation vector overlays (which also apply to 2-D NCAR/PSPLOT dumps) are controlled by namelist `veccon` (§2.4.9).

One movie frame per variable per vantage point per input HDF file is created. As described in §2.3.3, these files have file names of the form `rR**`*nnn*`id.`*vvv*`.f`, and may be in one of three formats:

1. "movie" format (`.f=.m`): unformatted `real*4` files used by subroutine `MAKEMPG` at the end of the job (`radfmt=1`, default);

2. "raw" format (`.f=` blank): MLN's `character*1` unformatted files (`radfmt=2`); and

3. HDF (`.f=.h`): (`radfmt=4`).

Multiple file formats are possible by specifying the appropriate binary addition. Thus, `radfmt=3=1+2` ⇒ both movie and raw format files are written.

In principle, many hundreds of files can result from a single *RADIO* run. Only "movie" format files (`radfmt=1`) are processed at the end of a *RADIO* run so that all files of the same variable and HDF input file but at different vantage points are strung together as an mpeg and then compressed into a tarball, yielding just two files per variable per input HDF file

(see §2.3.3). For all other formats, files generated at the different vantage points remain as separate file to be managed manually.

The bulk of cpu used by *RADIO* is used in setting up the lines-of-sight; the additional cpu required for each variable is nominal, and the only real "cost" of selecting all variables is the additional disc space required for the files.

All but the last parameters (`tarrad`) are identical to those found in the namelist `radcon` in *ZEUS-3D*.

```
parameter                     description                          default

radvar  (niov)  character*2 names for variables to be dumped
                as movie frames.  Legal mnemonics are:
                'A ' => pol. position angle [ 0.5 * atan (Q/U) ]
                'AV' => A with polarisation vectors superposed
                'F ' => fractional pol'n (fpol) raster plot
                'FV' => fpol with polarisation vectors
                'I ' => total intensity (toti) raster plot
                'IV' => toti with polarisation vectors
                'P ' => polarised intensity (poli) raster plot
                'PV' => poli with polarisation vectors
                'V ' => pol'n vector raster plot (black on white)
                'VR' => pol'n vector raster plot (white on black)
                'D ' => column density
                'P1' => column pressure (int. energy density)
                'T ' => column temperature (specific int. energy)
                'B ' => column magnetic field strength
                'SH' => column velocity shear
                'VV' => column divergence of velocity
                'BR' => bremsstrahlung emission
                'W ' => vorticity
                'M ' => Sonic Mach number
                'MA' => Alfven Mach number
                'MS' => Fast magnetosonic Mach number
                'MF' => Fast magnetosonic Mach number
                'DB' => d * B     (K-S R A variable)
                'EB' => p1**2 * B (K-S R A variable)
                'DE' => d / p1**2 (K-S R A variable)
nlrad   (niov)  = 0 => store data                                    0
                > 0 => store log10(data), concentrating colours
                       at low end. Dyn. range = nlrad, 1 => 100.
                < 0 => store log10(data), concentrating colours
                       at high end. Dyn. range =-nlrad, -1 =>-100.
radmin  (niov)  value to be assigned the minimum colour.             0.0
radmax  (niov)  value to be assigned the maximum colour.             0.0

Reversing the palette (nlrad<0) is useful for images in which radmin<0 and
radmax<0 (e.g., negative velocity divergences where it may be desirable to
have the "maximum" colour correspond to the minimum pixel value which has the
greatest absolute value).

radmm   (niov)  = 0 => use input "radmin", "radmax" for each image   1
                       If radmin=radmax=0, "radmm" reset to 1
                = 1 => compute "radmin", "radmax" for each image
                = 2 => extreme of computed "radmin", "radmax" used
                       for all frames (movie format only; for raw
                       and HDF formats, radmm=1, 2 are the same)
ncrad           number of colour contours in image                 255
```

```
radbox          = 0 => no box drawn around integration region       2
                = 1 => box lines drawn with highest colour
                =-1 => box lines drawn with lowest colour
                = 2 => all lines except those to "hidden corner"
                       are drawn with highest colour
                =-2 => all lines except those to "hidden corner"
                       are drawn with lowest colour
radfmt          = 1 => RADDMP creates 4-byte movie frames           1
                = 2 => RADDMP creates 1-byte raw pixel dumps.
                = 3 => RADDMP creates movie and raw files (1+2).
                = 4 => RADDMP creates 4-byte HDF files.
                = 5 => RADDMP creates movie and HDF files (1+4).
                = 6 => RADDMP creates raw and HDF files (2+4).
                = 7 => RADDMP creates movie, raw, and HDF files
                       (1+2+4).
                = 0 => no radio dumps are generated.
radpal  (niov)  = 0 => use user palette 'pfilerad'                  2
                = 1 => greyscale palette.
                = 2 => ZEUS' spectral colour palette.
pfilerad(niov)  character*128 user's palette file (radpal = 0).     2
                If file not found, radpal reverts to default.

Note that a different palette may be specified for each variable.

fraterad        FFmpeg's integer parameter.  Animation assembled   24
                to run at "fraterad" frames per second
mpgrad          = 1 => animations written in mpg format             4
                = 4 => animations written in mp4 format
                else=> mpegs not created, either because FFmpeg
                       not on system, or only one .m file was
                       created.
ppmrad          = 1 => .ppm files retained on exit (radfmt=1)       0
                = 0 => not
tarrad          = 1 => .m files (movie format; radfmt=1) gathered   1
                       into a g-zipped tar file.
                = 0 => not

Defaults for mpgrad, ppmrad, and tarrad are 0, 1, and 0 respectively if
only one movie frame (.m file) is requested.

       namelist / radcon   /
     1              radvar  , nlrad   , radmin  , radmax  , radmm
     2            , ncrad   , radbox  , radfmt  , radpal  , pfilerad
     3            , fraterad, mpgrad  , ppmrad  , tarrad
```

3. *ANIM8Z* User Manual

3.1 Introduction, Version 1.0

ANIM8Z is a stand-alone program that allows the user to modify animations generated by either *ZEUS-3D* or *RADIO*, and comes as part of the *ZEUS-3D* (version 3.6) public release tar-file `dzeus36.tar` (but not the abbreviated version `dz36.tar`). It starts by unpacking the specified movie tarballs (containing files in "movie format"; see `dzeus36_man.pdf`), converts them to "`PPM` format" using the specified palette along with the bracketing and logarithmic compression specified in the *ANIM8Z* input deck, `inanim8z`, and finishes by assembling the `PPM` files into a single `mp4` (or `mpg`) file.

The executable for *ANIM8Z* is created in an entirely analogous way as that for *ZEUS-3D*. The program is precompiled, compiled, loaded, and linked by use of the *script file* `anim8z10.s` (described below), and is managed by the precompiler *EDITOR*. It is assumed the reader has already installed *ZEUS-3D* and *EDITOR* on their unix platform, is familiar with their use, and is looking for a way to modify the animations that have been generated by a *ZEUS* or *RADIO* run. Therefore, the reader who has not yet installed *ZEUS-3D* on their unix platform should do so first by downloading `dzeus36.tar` from zeus3d.ca/zeus3d and following the instructions found therein.

3.2 Running *ANIM8Z*

3.2.1 Installation

This subsection describes how to install *ANIM8Z* on your *UNIX*-based system, and assumes that your platform already supports the widely-available mpeg-encoder, `FFmpeg`. If it does not, you can download it and its documentation at `ffmpeg.org`.

Having already downloaded `dzeus36.tar` from zeus3d.ca/zeus3d and unpacking the tarball, go to the directory `zeus_v3.6_ICA` and type `ls -FC`. On your screen you should see:

```
README          editor/         manuals/        radio/          zeus/
anim8z/         examples/       nmlst/          sci/
bldlib2*        grfx/           plotz/          testz/
```

In addition to the files in the directory `anim8z/`, the *EDITOR* executable `editor/xedit22` and the library `nmlst/namelist.a` (both created during the installation of *ZEUS-3D*) are needed to create the *ANIM8Z* executable, `xanim8z10`.

Go to the `anim8z` directory and once again type `ls -FC` where you'll find:

```
README.anim8z   anim8z10        anim8z10.s      movies/         zRB_za.tar.gz
```

The file `anim8z10` is the source code for *ANIM8Z*, version 1.0, `anim8z10.s` is the script file used to compile `anim8z10` and described below in §3.2.2, and `zRB_za.tar.gz` is a datafile used to exemplify the use of *ANIM8Z*. The `mp4` files in directory `movies` are for the example in §3.2.3

As is true for *ZEUS-3D* and *EDITOR*, *ANIM8Z* specifically supports seven compilers: `cf77` (Cray), `f90` (Sun), `g95` (`g95.org`), `gfortran` (Gnu), `ifort` (Intel), `pgf77` (Portland Group), and `xlf` (IBM). If one of these compilers is available on the user's platform, go to step 4.

Step 1. If none of these compilers is available, then suppose the *FORTRAN* compiler that is available is called `newfor`. Open `anim8z10` in a text editor, replace all occurrences of `GFORTRAN` (note all-caps) with `GFORTRAN.or.NEWFOR`, and save the changes.

Step 2. Open the file `anim8z10.s` in a text editor, replace the one occurrence of `setenv FCOMP gfortran` with `setenv FCOMP newfor` (note no caps), and save the change.

Step 3. At your unix prompt, type: `csh -v anim8z10.s`

If this succeeds in creating the executable `xanim8z10`, installation is complete. If not, it will likely fail by citing calls to unknown implicit functions such as `cpu_time`, `date_and_time`, and/or `system` used by *ANIM8Z* subroutines `MAKEMPG`, `CCLOCK`, and `WCLOCK`. For many compilers such as `gfortran`, these implicit functions do the following:

`cpu_time`	returns the cpu time since the beginning of the run in seconds;
`date_and_time`	returns eight integers giving the year, month, day, *etc.* Search for `date_and_time` in `anim8z10` for details.
`system`	executes a systems call (such as removing a disc file) at run time.

You will have to find the equivalents to these functionalities for `newfor`. Once determined, open up `anim8z10` again, find the offending implicit functions, and make the necessary changes.

Step 4. At your unix prompt, type: `csh -v anim8z10.s`

For those following steps 2 and 3, the complaints about the foreign system calls should disappear. If the executable `xanim8z10` is created, installation is complete. If not, some troubleshooting will be necessary for which little general guidance can be given.

3.2.2 The script file `anim8z10.s`

The script file `anim8z10.s` used in the previous subsection to install *ANIM8Z* is similar to (and somewhat simpler than) the script file `dzeus36.s` used to install *ZEUS-3D*. As of this writing, the script file `anim8z10.s` is as follows:

```
#---+----1----+----2----+----3----+--+----3----+----2----+----1----+---#
#============= SCRIPT FILE TO CREATE THE ANIM8Z EXECUTABLE ============#
#
#----------------------------------------------> Select FORTRAN compiler.
#  Supported compilers include:
#     cf77 (Cray), f90 (Sun), g95 (g95.org), gfortran (Gnu), ifort
#     (Intel), pgf77 (Portland Group), xlf (IBM)
#
setenv FCOMP gfortran
#========================================> Get files from home directory.
if(! -e ./xedit22) cp ../editor/xedit22 .
```

```
 12  if(! -e ./namelist.a) cp ../nmlst/namelist.a .
 13  #=======================> If necessary, create the directory "anim8z1.0".
 14  if(! -e ./anim8z1.0) mkdir ./anim8z1.0
 15  #----------------------------------------------> Create the change deck.
 16  rm -f ./chganim8z
 17  cat << EOF > chganim8z
 18  *define $FCOMP
 19  *d par.13
 20         parameter     ( nin  = 20, nxpx = 1000, nypx = 1000 )
 21  EOF
 22  #=======================> Create the input deck for EDITOR, and execute.
 23  rm -f ./inedit
 24  cat << EOF > inedit
 25   \$editpar   inname='anim8z10', chgdk='chganim8z', idump=1, job=3
 26             , ipre=1, inmlst=1, iupdate=1, iutask=0, ibanner=0
 27             , safety=0.50, branch='anim8z1.0', xeq='xanim8z10'
 28             , makename='makeanim8z', compiler='$FCOMP'
 29  c           , coptions='debug'
 30             , coptions='optimise'
 31             , libs='namelist.a'                                        \$
 32  EOF
 33  chmod 755 ./xedit22
 34  ./xedit22
 35  #-----------------------------------> Create the input deck for ANIM8Z.
 36  rm -f ./inanim8z
 37  cat << EOF > inanim8z
 38   \$animcon  mpgfmt=4, infile='zRB_za.tar.gz', infmt=2, pal=1, colmm=1
 39            , nlog=100                                                  \$
 40  EOF
 41  #==========================================> MAKE the ANIM8Z executable.
 42  make -f ./makeanim8z
 43  #=====================================================> Tidy up directory.
 44  rm -f ./editlp ./inedit ./output ./xedit22 ./chganim8z ./chganim8z.m ./*.a
./anim8z10.m ./anim8z10.f
```

As with the *ZEUS-3D* script file `dzeus36.s`, the *ANIM8Z* script file is divided into several segments heralded either by a double line (`====`) or a single line (`----`). Segments heralded by a double line are those which once set should rarely, if ever, be modified. Those heralded with a single line are segments that may have to be modified every time the program is used.

On line 9 of the first segment, the desired *FORTRAN* compiler is selected. Here, `FCOMP` is a "system variable" set by the *UNIX* `setenv` command, and is used in two places. First, on line 18 it serves as the *EDITOR* macro to tailor `anim8z10` to the chosen compiler. Second, on line 28 it is the setting for the namelist parameter `compiler` in the input deck `inedit` which determines which compiler the makefile uses to compile `anim8z10`.

Lines 11 and 12 of the second segment retrieve the files necessary to create the *ANIM8Z* executable and are retrieved only if they do not already exist in the present working directory (`pwd`) [`if (! -e` *filename*`)`]. This example assumes that the script file is launched from the directory `anim8z` created when `dzeus36.tar` from zeus3d.ca/zeus3d is unpacked. Thus, the file `anim8z10` (containing the source code) is already in this directory, and not retrieved. Files retrieved from other directories include:

`xedit22`	preprocessor executable
`namelist.a`	single precision library of subroutines to enable namelist feature

Line 14 of the third segment creates the directory `anim8z1.0` into which all files split from the master source file `anim8z10` by *EDITOR* and object files once compiled are placed.

The fourth segment (lines 15–21) generates the change deck `chganim8z` which gets merged with the master file `anim8z10` by *EDITOR* during the precompilation step. Line 18 sets the compiler *EDITOR* will put in the makefile used to compile the code once precompilation is complete. Lines 19 and 20 set the array dimensions suitable for the example described in §3.2.3. See §3.2.4 for a discussion on what each parameter means.

The fifth segment (lines 22–34) generates the namelist input data file `inedit` which the *EDITOR* executable `xedit22` uses to precompile `anim8z10`. In this example, the namelist parameters are set so that `chganim8z` will first be merged into `anim8z10` which is then precompiled (since `ipre=1`) for `gfortran` with the namelist feature *on* (`inmlst=1`), the conditional splitting feature *on* (`iupdate=1`), and the micro-tasking feature *off* (`iutask=0`). All files split from the merged master source code will be placed in the directory `anim8z1.0` with the extension `.f`, appropriate for *FORTRAN* source code. Since `iupdate=1` *and* `makename` is set to a non-blank character string in the input deck, a makefile with the name `makeanim8z` will be generated. The makefile will use the compiler `gfortran -c` and loader `gfortran` with the highest safe optimisation (`coptions='optimise'` not commented out). The library `namelist.a` will be linked with the *ANIM8Z* object code to generate a binary executable called `xanim8z10`.

Finally in this segment, line 34 launches `xedit22` and `anim8z10` is precompiled.

The sixth segment (lines 35–40) sets the namelist input deck, `inanim8z`, used by *ANIM8Z* itself to generate the desired output from the specified input tarball file(s). All namelist parameters for *ANIM8Z* are described in §3.4.

Line 42 in the seventh segment fires up the makefile `makeanim8z` generated in the fifth segment. The result of this step is the executable, `xanim8z10`.

Finally, line 44 of the last segment removes some of the intermediate files no longer needed.

3.2.3 A quick example using *ANIM8Z*

If `anim8z10.s` executed successfully in the previous subsection, there should be an input deck `inanim8z` (lines 38 and 39 in `anim8z10.s` described above, without the backslashes before the $ signs) and an executable `xanim8z10` in your `pwd`. These two files plus the data file `zRB_za.tar.gz` are all that are needed to run the example provided herein.

So, remaining within the `anim8z` directory, type `xanim8z10` at the unix prompt, and *ANIM8Z* will fire up, making the animation requested in `inanim8z` from the movie tarball file `zRB_za.tar.gz` created by *ZEUS-3D*. If your platform does not support the mpeg-encoder `FFmpeg`, this step will fail. In this stead and for the time-being, you can look at the expected output in the directory `movies`.

The resulting animation (called `mRB_za.mp4`) consists of the first eleven frames of a low resolution (128^3) super-Alfvénic turbulence run, similar to the run used to generate the HDF file used to exemplify *PLOTZ* (§1.2.3). The format is mpeg-4 which can be viewed by a number of mpeg-viewers (*e.g.*, *VLC*), and shows the rapid development of filaments in the magnetic field energy density. The rendering is a line-of-sight integration through the datacube, produced by the in-line *RADIO* feature in *ZEUS-3D*.

All namelist parameters for *ANIM8Z*, including those set in lines 38 and 39 of the script file `anim8z10.s` used for this example (§3.2.2), are described in detail in §3.4.

3.2.4 *ANIM8Z* parameters

There are only three parameters of which one must be aware when running *ANIM8Z*, all of which are listed in line 20 in the script file `anim8z10.s` in §3.2.2. These are:

parameter	interpretation	default
nin	maximum number of input movie tarballs	20
nxpx	maximum number of pixels in x-direction of image plane	1000
nypx	maximum number of pixels in y-direction of image plane	1000

If the user specified a tarball whose movie frames have dimensions larger than either or both `(nxpx,nypx)`, an error message is issued and execution is terminated.

3.3 Output from *ANIM8Z*

ANIM8Z generates only one type of output file: mpegs in either `.mpg` or `.mp4` format.

namelist: `animcon`

filename: `m`*t*`**id`*.mm*, where `m` is the common prefix to all 1-D plot files generated by *ANIM8Z*, *t* is parsed from the tarball filename and is a one-character indicator of whether the tarball consists of 2-D pixel dumps (`t=i`) or *RADIO* integrations (`t=R`), `**` is a two-character indicator of the variable, `id` is the two-character ID tag also parsed from the tarball filename, and *mm* is a two-digit extension indicating the slice number (for `t=i` only).

format: mpg—old mpeg format
mp4—more recent mpeg format that is better compressed and can be read by more viewers

3.4 Namelist parameters for *ANIM8Z*

There is only one namelist for *ANIM8Z*, and many of the namelist parameters therein are similar to those found in *ZEUS-3D* for the same purpose. The primary difference in parameter naming conventions is that in *ZEUS-3D*, there needs to be two sets of parameters—one for the 2-D pixel dumps, another for the line-of-sight RADIO dumps—and each of these have either `pix` or `rad` as part of their name. Here, no distinction between pixel and *RADIO* dumps is necessary, and the parameter names are more generic.

As in the *ZEUS* manual, a discussion of what each namelist parameter controls is given, followed by the namelist statement that appears in *ANIM8Z*. For a reminder of the nature of namelists and how they are used in the *ZEUS* family of codes, the reader is referred to the first page of Appendix B in the `dzeus36` user manual.

`ANIMCON`—`ANIM`ation `CON`trol (defined in subroutine `nmlsts`)

This namelist controls which movie tarballs are to be reprocessed, and how the reprocessing is to be done. Movie tarball filenames have the format `pt**id.mm.tar.gz`, where `p` is either `z` for files created by *ZEUS-3D* or `r` for files created by *RADIO*, *t*, `**`, `id`, and *mm* are as described above (§3.3), and the suffix `.tar.gz` indicates this is a `gzipped` tarball. One can specify as many files as needed (up to `nin`; see §3.2.4), each with their own palette, bracketing, logarithmic compression, mpeg quality factors, *etc.* In future releases, one will be able to ask that certain files be spliced together, or appear as separate animations in the same frame. Annotations of the final mpeg files will also be possible.

```
parameter                       description                          default

I/O control

infile    (nin)  character*64 name(s) of movie tarball(s) to be
                 rebracketed and/or assembled into another movie.
infmt            format of movie files in input tarball                3
                 = 1 => qmn, qmx
                        nxz, nyz
                        ( xa(nx), nx=1,nxz+1 )
                        ( ya(ny), ny=1,nyz+1 )
                        ( ( qty(nx,ny), nx=1,nxz ), ny=1,nyz )
                 = 2 => qmn, qmx
                        nxz, nyz
                        th0, ph0
                        ( xa(nx), nx=1,nxz+1 )
                        ( ya(ny), ny=1,nyz+1 )
                        ( ( qty(nx,ny), nx=1,nxz ), ny=1,nyz )
                 = 3 => qmn, qmx
                        nxz, nyz, nhy
                        th0, ph0, prtime
                        ( xa(nx), nx=1,nxz+1 )
                        ( ya(ny), ny=1,nyz+1 )
                        ( ( qty(nx,ny), nx=1,nxz ), ny=1,nyz )

Frame control

iflip     (nin)  = 0 => no flipping of images                          0
                 = 1 => image flipped about x-axis before writing
                        (size of image not changed, just flipped)
jflip     (nin)  = 0 => no flipping of images                          0
                 = 1 => image flipped about y-axis before writing
irefl     (nin)  = 0 => no reflection about x-axis                     0
                 = 1 => image reflected about x-axis on output
                        => twice the number of y-pixels requested
jrefl     (nin)  = 0 => no reflection about y-axis                     0
                 = 1 => image reflected about y-axis on output
                        => twice the number of x-pixels requested
nlog      (nin)  = 0 => store data                                     0
                 > 0 => store log10(data), concentrating colours
                        at low end.  Dyn. range = nlog, 1 => 100.
                 < 0 => store log10(data), concentrating colours
                        at high end. Dyn. range =-nlog, -1 => -100.
colmin    (nin)  value to be assigned the minimum colour.              0.0
colmax    (nin)  value to be assigned the maximum colour.              0.0
colmm     (nin)  = 0 => use "colmin", "colmax" for each image          0
                        If colmin=colmax=0, "colmm" set to 1
                 = 1 => set "colmin", "colmax" to image extrema
```

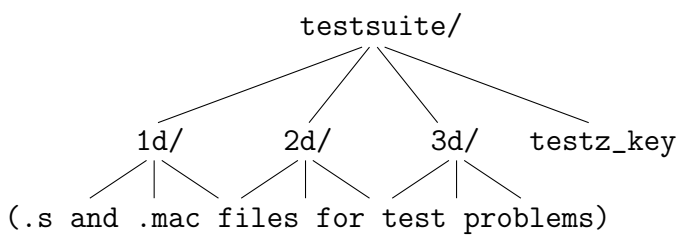
```
                = 2 => set "colmin", "colmax" to extrema of all
                       images in tarball
nccon    (nin)  number of colour contour levels in each image      255

Movie control

pal      (nin)  = 0 => use user palette "pfile"                      2
                = 1 => greyscale palette.
                = 2 => spectral colour palette.
pfile    (nin)  character*64 user's palette file (pal = 0).  If
                file not found, "pal" reverts to default (2).
frate    (nin)  FFmpeg's integer parameter.  Animation assembled    24
                to run at "frate" frames per second
mpgfmt          = 4 => animations written in mp4 format              4
                else=> mpg format
kpppm           = 0 => PPM files are discarded, temp. directory      0
                       anim8z.dir deleted after use
                = 1 => PPM files are retained in anim8z.dir

       namelist / animcon   /
     1               infile  , iflip   , jflip   , irefl   , jrefl
     2             , nlog    , colmin  , colmax  , colmm   , nccon
     3             , pal     , pfile   , frate   , mpgfmt  , kpppm
```

4. *TESTZ* User Manual

4.1 Introduction, Version 1.0

TESTZ is a stand-alone program that runs *ZEUS-3D* through its extensive *test suite* of > 200 tests, and comes as part of the *ZEUS-3D* (version 3.6) public release tar-file `dzeus36.tar` (but not the abbreviated version `dz36.tar`). The *TESTZ* executable is created in an entirely analogous way as that for *ZEUS-3D*. The program is precompiled, compiled, loaded, and linked by use of the *script file* `testz10.s` (§4.2.2), and is managed by the *precompiler EDITOR*. It is assumed the reader has already installed *ZEUS-3D* and *EDITOR* on their unix platform, is familiar with their use, and is now looking to test *ZEUS-3D* using the established test suite, or even add tests of their own. Therefore, the reader who has not yet installed *ZEUS-3D* on their unix platform should do so first by downloading `dzeus36.tar` from `zeus3d.ca/zeus3d` and following the instructions found therein.

TESTZ is designed to run *ZEUS-3D* sequentially through each of the test problems (executed in an order to minimise the frequency with which *ZEUS-3D* has to be recompiled) and report if any of the test results differ from those from the last time the test suite was run. Each test problem has its own `.s` and `.mac` file[3] in the directory `zeus_v3.6_ICA/testz/testsuite` with the following sub-tree:



The test problems are organised according to their dimensionality, and the names of the subdirectories and the files contained therein should not be altered in any way. The (ascii) file `testz_key` is the master key file maintained automatically by *TESTZ* containing a list of all the tests found in the subdirectories `1d`, `2d`, and `3d`, along with values for the two *key comparators* used to determine whether any of the test results change from one run to the next. It too should not be altered by the user, with the only exception being if new tests are added to the test suite (see §4.2.3).

Listed below are the first three lines (after the opening banner and comments) of the file `testz_key` as delivered in the `dzeus36.tar` tarball.

```
ryujones_fig1a-x1-trnvrsn0          aa   1   1278  1.004760960432550E-06  r
ryujones_fig1a-x1                   ab   1   1279  2.157984828199615E-05  r
ryujones_fig1a-x1-trnvrsn2          ac   1   1279  2.157984828195452E-05  r
```

[3]These would be familiar to those who have already installed *ZEUS-3D* on their system and run the sample problem included in the installation instructions.

Each entry in `testz_key` includes the name of the test followed by five data. The first datum is a two-character "problem tag" assigned to each problem to distinguish its output (*e.g.*, graphics) from that of the others. The second datum is a single-character digit indicating the dimensionality of the problem, and thus in which directory its `.s` and `.mac` files are found. The third datum and the first key comparator is an up-to-six-digit integer indicating the number of MHD cycles ("`nhy`") required by the test problem the last time it was run. The fourth datum and the second key comparator is the value to sixteen significant figures of the last time step ("`last dt`") taken by the test problem the last time it was run. Finally, the last datum is a single character flag indicating whether that test was run ("`r`") or skipped ("`s`") the last time the test suite was run through.

Each time *TESTZ* is used in its "run-mode" (see §4.2.3), each test specified by the user is run in the same order as they appear in the file `testz_key`. To "run a test", *TESTZ* copies the appropriate `.s` and `.mac` files from the directory `testsuite` to the "scratch" directory `testz_scr` created by *TESTZ* where they are renamed to `dzeus36.s` and `zeus36.mac`. These files are then used to compile and link *ZEUS* in the usual way, and the executable so produced is run within the scratch directory with the input deck generated from `dzeus36.s`. When execution stops, the values of "`nhy`" and "`last dt`" are compared to those stored in the master file `testsuite/testz_key`. Any difference in "`nhy`" and any difference in "`last dt`" greater than a user-specified tolerance are reported both to the screen and to an ascii log-file, `testz_log`. Such differences, if any, may either be understood and accepted as a consequence of the changes made to the code, or point to problems within the code that must be fixed.

Once all differences are understood and accepted, *TESTZ* may be used in its "save-mode" where the new key file (`testz_key.new`) overwrites the previous one (`testsuite/testz_key`) and then is copied to a dated file in the "archive" directory created automatically by *TESTZ* (if it doesn't already exist) in the `./testsuite` directory. Further and if the user wishes, all `.ps` and `.mp4` files generated by the test problems can be archived in a dated "`dump`" directory which then, if desired, can be compressed into a tarball. Together, these graphics files require about 1 GByte of disc space, 150 Mbytes compressed.

4.2 Running *TESTZ*

4.2.1 Installation

To install *TESTZ* on your *UNIX*-based system, go to the directory `zeus_v3.6_ICA` and type `ls -FC`. On your screen you should see:

```
README          editor/         manuals/        radio/          zeus/
anim8z/         examples/       nmlst/          sci/
bldlib2*        grfx/           plotz/          testz/
```

In addition to the files in the directory `testz/`, the *EDITOR* executable `editor/xedit22` and the library `nmlst/dnamelist.a` (both created during the installation of *ZEUS-3D*) are needed to create the *TESTZ* executable, `xtestz10`.

Go to the `testz` directory and once again type `ls -FC` where you'll find:

```
README.testz    testsuite/      testz10         testz10.s       xtz10*
```

The file `testz10` is the source code for *TESTZ*, version 1.0, `testz10.s` is the script file used to compile `testz10` (see §4.2.2), and `xtz10` is a command-line executable script file which can be used to run *TESTZ* in its four intended "modes" (see §4.2.3).

As is true for *ZEUS-3D* and *EDITOR*, *TESTZ* specifically supports seven compilers: `cf77` (Cray), `f90` (Sun), `g95` (`g95.org`), `gfortran` (Gnu), `ifort` (Intel), `pgf77` (Portland Group), and `xlf` (IBM), although the compiler dependence in the code is very slight. If one of these compilers is available on the user's platform, go to step 4.

Step 1. If none of these compilers is available, then suppose the *FORTRAN* compiler that is available is called `newfor`. Open `testz10` in a text editor, replace all occurrences of `GFORTRAN` (note all-caps) with `GFORTRAN.or.NEWFOR`, and save the changes.

Step 2. Open the file `testz10.s` in a text editor, replace the one occurrence of `setenv FCOMP gfortran` with `setenv FCOMP newfor` (note no caps), and save the change.

Step 3. At your unix prompt, type: `csh -v testz10.s`

If this succeeds in creating the executable `xtestz10`, installation is complete. If not, it will likely fail by citing calls to unknown implicit functions such as `cpu_time`, `date_and_time`, and/or `system` used by *TESTZ* subroutines `IXEQOS` and `WCLOCK`. For many compilers such as `gfortran`, these implicit functions do the following:

`cpu_time`	returns the cpu time since the beginning of the run in seconds;
`date_and_time`	returns eight integers giving the year, month, day, *etc.* Search for `date_and_time` in `testz10` for details.
`system`	executes a systems call (such as removing a disc file) at run time.

You will have to find the equivalents to these functionalities for `newfor`. Once determined, open up `testz10` again, find the offending implicit functions, and make the necessary changes.

Step 4. This step is necessary only if the user wishes to run *TESTZ* and/or *ZEUS* in directories other than those set up by unpacking the `dzeus36.tar` tarball.

Open the source file `testz10` in a text editor, and scroll down to the line:

```
c====USER ALERT===USER ALERT===USER ALERT===USER ALERT===USER ALERT===USER ALERT=====
```

As indicated in the comments following, it will be necessary to change the absolute or relative paths to the seven key files needed by *TESTZ* to compile and load `dzeus36` in order to do the tests. Once this segment of code is found, the required changes should be obvious.

Step 5. At your unix prompt, type: `csh -v testz10.s`

For those following steps 2 and 3, the complaints about the foreign system calls should disappear. If the executable `xtestz10` is created, installation is complete. If not, some troubleshooting will be necessary for which little general guidance can be given.

4.2.2 The script file `testz10.s`

The script file `testz10.s` used in the previous subsection to install *TESTZ* is similar to (and somewhat simpler than) the script file `dzeus36.s` used to install *ZEUS-3D*. As of this writing, the script file `testz10.s` is as follows:

```
 1  #---+----1----+----2----+----3----+--+----3----+----2----+----1----+---#
 2  #============= SCRIPT FILE TO CREATE THE TESTZ EXECUTABLE =============#
 3  #                                                                      #
 4  #----------------------------------------------> Select FORTRAN compiler.
 5  #  Supported compilers include:
 6  #     cf77 (Cray), f90 (Sun), g95 (g95.org), gfortran (Gnu), ifort
 7  #     (Intel), pgf77 (Portland Group), xlf (IBM)
 8  #
 9  setenv FCOMP gfortran
10  #========================================> Get files from home directory.
11  if(! -e ./xedit22) cp ../editor/xedit22 .
12  if(! -e ./dnamelist.a) cp ../nmlst/dnamelist.a .
13  #=======================> If necessary, create the directory "testz1.0".
14  if(! -e ./testz1.0) mkdir ./testz1.0
15  #-----------------------------------------------> Create the change deck.
16  rm -f ./chgtestz
17  cat << EOF > ./chgtestz
18  *define $FCOMP
19  *d tzcom.7
20         parameter     ( ntst = 256 )
21  EOF
22  #=======================> Create the input deck for EDITOR, and execute.
23  rm -f ./inedit
24  cat << EOF > inedit
25   \$editpar   inname='testz10', chgdk='chgtestz', idump=1, job=3, ipre=1
26            , inmlst=1, iupdate=1, iutask=0, ibanner=0, safety=0.50
27            , branch='testz1.0', xeq='xtestz10', makename='maketestz'
28            , compiler='$FCOMP', coptions='optimise'
29            , libs='dnamelist.a'                                        \$
30  EOF
31  chmod 755 ./xedit22
32  ./xedit22
33  #-------------------------------------> Create the input deck for TESTZ.
34  rm -f ./intestz
35  cat << EOF > intestz
36   \$tzcon    iotty=6, iolog=10, tol=1.0e-7, verbose=0, rors=1, keepio=0
37           , compiler='gfortran', tests='*'                             \$
38  EOF
39  #=========================================> MAKE the TESTZ executable.
40  make -f ./maketestz
41  #====================================================> Tidy up directory.
42  rm -f ./editlp ./inedit ./output ./xedit22 ./checkin.o ./chgtestz ./chgtes
tz.m ./testz10.f ./testz10.m ./maketestz ./*.a
```

As with the *ZEUS-3D* script file `dzeus36.s`, the *TESTZ* script file is divided into several segments heralded either by a double line (`====`) or a single line (`----`). Segments heralded by a double line are those which once set should rarely, if ever, be modified. Those heralded with a single line are segments that may have to be modified every time the program is used.

On line 9 of the first segment, the desired *FORTRAN* compiler is selected. Here, `FCOMP` is a "system variable" set by the *UNIX* `setenv` command, and is used in two places. First,

on line 18 it serves as the *EDITOR* macro to tailor `testz10` to the chosen compiler. Second, on line 28 it is the setting for the namelist parameter `compiler` in the input deck `inedit` which determines which compiler the makefile uses to compile `testz10`.

Lines 11 and 12 of the second segment retrieve the files necessary to create the *TESTZ* executable and are retrieved only if they do not already exist in the present working directory (`pwd`) [`if (! -e` *filename*`)`]. This example assumes that the script file is launched from the directory `testz` created when `dzeus36.tar` from zeus3d.ca/zeus3d is unpacked. Thus, the file `testz10` (containing the source code) is already in this directory, and not retrieved. Files retrieved from other directories include:

`xedit22`	preprocessor executable
`dnamelist.a`	double precision library of subroutines to enable namelist feature

Line 14 of the third segment creates the directory `testz1.0` into which all files split from the master source file `testz10` by *EDITOR* and object files once compiled are placed.

The fourth segment (lines 15–21) generates the change deck `chgtestz` which gets merged with the master file `testz10` by *EDITOR* during the precompilation step. Line 18 sets the compiler *EDITOR* will put in the makefile used to compile the code once precompilation is complete. Lines 19 and 20 set the array dimension `ntst`, the maximum number of tests that the user may specify. Note that with the use of wildcards (see §4.5), `ntst` can be significantly less than the total number of tests in the test suite.

The fifth segment (lines 22–32) generates the namelist input data file `inedit` which the *EDITOR* executable `xedit22` uses to precompile `testz10`. In this example, the namelist parameters are set so that `chgtestz` will first be merged into `testz10` which is then precompiled (since `ipre=1`) for `gfortran` with the namelist feature *on* (`inmlst=1`), the conditional splitting feature *on* (`iupdate=1`), and the micro-tasking feature *off* (`iutask=0`). All files split from the merged master source code will be placed in the directory `testz1.0` with the extension `.f`, appropriate for *FORTRAN* source code. Since `iupdate=1` *and* `makename` is set to a non-blank character string in the input deck, a makefile with the name `maketestz` will be generated. The makefile will use the compiler `gfortran -c` and loader `gfortran` with the highest safe optimisation (`coptions='optimise'`). The library `dnamelist.a` will be linked with the *TESTZ* object code to generate a binary executable called `xtestz10`.

Finally in this segment, line 32 launches `xedit22` and `testz10` is precompiled.

The sixth segment (lines 33–38) sets the namelist input deck, `intestz`, used by *TESTZ* itself to run the *ZEUS-3D* tests or to save the various files generated by the tests. All namelist parameters for *TESTZ* are described in §4.5.

Line 40 in the seventh segment fires up the makefile `maketestz` generated in the fifth segment. The result of this step is the executable, `xtestz10`.

Finally, line 42 of the last segment removes some of the intermediate files no longer needed.

4.2.3 Using *TESTZ* (`xtz10`)

If `testz10.s` executed successfully in the previous subsection, there should be an input deck `intestz` (lines 36 and 37 in `testz10.s` described above, without the backslashes before the $ signs) and an executable `xtestz10` in your `pwd` which, for the purposes of this discussion, is

assumed to be in the `zeus_v3.6_ICA/testz` directory. These two files in the same directory that contains the test suite directory `testsuite` as delivered in the `dzeus36.tar` tar-file are all that are needed to run the program.

Now, while one can certainly use *TESTZ* by modifying the namelist parameters in the input file `intestz` as required then launching `xtestz10` (as is done for `xdzeus36` and the other ancillaries), the command line script file `xtz10` is provided to make this task even simpler as this will not require any modifications to the input deck `intestz`.

To use `xtz10`, one types at the unix prompt one of the following five commands:

1. `xtz10 run`

   This launches `xtestz10` to run all tests in the test suite which, on my MacBook Pro (2.4 GHz Intel core i7) takes just under an hour. Alternately, the user may launch a subset of the tests only. For example, if only the 1-D tests and the 3-D Sedov blasts tests are to be run, open `xtz10` in a text editor, scroll down to the line:

   ```
   if ($1 == run) then
   ```

   and then change the fifth line following from:

   ```
           , tests='*'                                                    \$
   ```

   to:

   ```
           , tests='1d','sed*'                                            \$
   ```

   Doing only the Ryu & Jones tests, for example, is accomplished by setting:

   ```
           , tests='ryu*'                                                 \$
   ```

   and so it goes.

2. `xtz10 rerun`

   Sometimes for whatever reason, some of the tests may fail to run, in which case they are skipped (generating a warning message to both the screen and log file, and writing an "`s`" in the last character of the test's entry in `testz_key.new`) and the next test is tried. Reasons why a test may be skipped include:

   - Either the `.s` file or `.mac` file were not found. Make sure the names of the files are spelled correctly and that they are in the anticipated (by *TESTZ*) directory (`testsuite/1d`, `2d`, or `3d`).
   - Compilation may have failed, in which case the warning message:

     ```
     TESTZ   : |---WARNING---> Skipping test: <test name>
     TESTZ   : Can't find "last dt" and "nhy" in file zeus3d_log
     ```

     would be issued on the screen and in the logfile `testz_log`. In this case, go to the scratch directory `testz_scr` generated by *TESTZ* (in which all compilation and execution of *ZEUS-3D* is done), and try running `dzeus36.s` for the troublesome test problem directly. It may be that a parameter in the `.mac` file got changed that inhibited compilation, or a namelist parameter in `inzeus` was misspelled, or changes to the code made by the user caused a compilation error.

Once reasons for tests being skipped are known and corrected, issuing `xtz10 rerun` causes *TESTZ* to run only the skipped tests without redoing the tests performed successfully. The new key file `testz_key.new` is updated (not overwritten), and should be as though the skipped tests were never skipped in the first place.

3. `xtz10 savekey`

   After the user is satisfied that all tests have run successfully and that all differences, if any, from the previous run of the test suite are understood and accepted as appropriate consequences of any changes made to the code, the user can elect to save just the new key and log files to the archive as well as overwrite the master key file (`testsuite/testz_key`) against which the next run through the test suite will be compared.

4. `xtz10 saveall`

   In addition to saving the key and log files and updating the master key file, this command saves all graphics files (ending in `.ps` and `.mp4`) generated by the test problems run. As delivered (with `keepio=2`; see §4.5), these are moved to a dated "`dump`" directory in the archive (*e.g.*, `testsuite/archive/dumps_yy-mm-dd`), and then compressed to a tarball (*e.g.*, `testsuite/archive/dumps_yy-mm-dd.tar`) occupying about 150 Mbytes. If the user wishes to forgo the compression step (and thus occupy ∼1 Gbyte for all files), change the value of `keepio` in the file `xtz10` to 1.

   There is no particular need for these files to be saved, and particularly not each time the test suite is run, unless the user wishes to peruse the images to gain a feel for the tests (not a bad idea at least *once* during a user's relationship with *ZEUS*!)

5. `xtz10 delete`

   This command will delete all files generated by the last run in case they are not to be saved to the archive. These include the directory `testz_scr` and all its contents, along with files `testz_key.new`, `testz_log`, and `xtestz10`.

Note that both `savekey` and `saveall` will delete the scratch space directory `testz_scr` on exiting. Thus, in the case of `savekey`, the graphics files will be deleted with the scratch directory. If the user then changes his or her mind after issuing the `savekey` command and wishes to save the graphics files after all, the only recourse is to run the test suite again (`xtz10 run`).

Finally, note that *TESTZ* "date-stamps" the key and log files as well as the data dump directories/tarballs, but does not "time-stamp" them. Thus, if the user were to run and save the results from the test suite more than once in a day, the results from the more recent run would overwrite those from the previous. Only results obtained on separate days are stored as separate archived files and directories.

4.2.4 *TESTZ* parameters

The only parameter of which one might be aware when running *TESTZ* is listed in line 20 in the script file `testz10.s` in §4.2.2:

parameter	interpretation	default
ntst	maximum number of tests user may specify in namelist array "tests"	256

Note with the use of wildcards ('*'), the default setting should be more than enough for any use; it is exceedingly unlikely anyone would ever wish to list specifically more than 256 tests in the namelist parameter `tests`! Should the user specify more tests than `ntst`, those in excess are simply ignored.

4.3 The test suite

As delivered in the tarball `dzeus36.tar`, there are 239 test problems in the test suite testing various algorithms, geometries, and features in the code in various directions, planes, and in full 3-D. These tests are described by category below, followed by instructions on how to add new tests to the test suite.

4.3.1 Advection tests

Advection is a special case of full (magneto)hydrodynamics in which it is assumed $\nabla \cdot \vec{v}$ and all *source terms* (such as ∇p) are zero. In this realm, the fluid equations act as simple wave equations in the *primitive variables* and, for the code to be able to perform advection properly, it must be able to move wave forms (*e.g.*, square pulses) at a constant speed through the grid without excessive diffusion and without exciting numerical instabilities, such as those triggered by CFL violations.

Advection (and relaxation) tests performed in all geometries and directions include:

- `advection*`: 1-D advection problems in Cartesian coordinates in each of the 1-, 2-, and 3-directions testing each of the `trnvrsn=0,1` algorithms ("legacy transport" and "finely interleaved transport" ≡ "FIT", respectively; see the *ZEUS-3D* user manual, `dzeus36_man.pdf`) as well as the various magnetic induction algorithms, namely `CMoC`, `HSMoC`, `MoC`, and `noMoC` (*ZEUS*' pre-MoC induction algorithm, CT; Hawley & Evans, 1988, ApJ, 332, 659). A square pulse in various primitive variables is advected with second and third order interpolation.

- `advect2d*`: 2-D advection problems in Cartesian coordinates in each of the 1-2, 2-3, and 3-1 planes, testing each of the `trnvrsn=0,1` algorithms. Previous to v3.6, *ZEUS* could not stably advect square pulses at any non-zero angle to the coordinate axes. The `trnvrsn=1` (FIT) algorithm, new to v3.6, is the first *ZEUS* transport algorithm to correct this problem.

- `advection-x1-rtp*`: 1-D advection problems in spherical polar coordinates in the 1-direction (r) toward the symmetry axis. With $v_1 \sim -x_1$, these tests test the predictions that $\rho, e_1 \sim e^{3\omega t}$, $B_2, B_3 \sim e^{2\omega t}$, and $v_2, v_3 \sim e^{\omega t}$, with $\omega = 1$. Levels of advected quantities fall a little short of the expected values because of incomplete time-centring. The fact that the v_1 and ρ profiles remain linear even at the axis is significant.

- `advection-x2-zrp*`: 1-D advection problems in cylindrical coordinates in the 2-direction (r) toward the symmetry axis. With $v_2 \sim -x_2$, these tests test the predictions that $\rho, e_1, B_1 \sim e^{2\omega t}$, $v_3, B_3 \sim e^{\omega t}$, and $v_1 \sim$ constant, with $\omega = 1$. Levels of advected quantities fall a little short of the expected values because of incomplete time-centring.

- `relaxation-x1*`: related to advection problems, relaxation problems (with $v_1 \sim +x_1$) move matter away from the symmetry axis and monitor constant levels in all variables rather than pulses. Here, relaxation in the 1-direction in spherical polar geometry is tested where it is significant that the constant levels fall off exponentially as expected (modulo the time centring problem mentioned above), and maintain a perfectly flat profile all the way into the symmetry axis.

- `relaxation-x2*`: relaxation in cylindrical coordinates in the 2-direction; see comments in the item above.

4.3.2 Alfvén wave propagation

Alfvén wave tests study a subset of the full dynamical equations in which only $v_\perp$ and $B_\perp$ vary. Historically, this test was introduced to demonstrate Stone and Norman's *Method of Characteristics* algorithm (MoC; 1992, ApJS, 80, 791), a precursor to the fully upwinded Godunov schemes used in many MHD algorithms (but not *ZEUS*) that followed.

The test suite includes three flavours of the Alfvén wave test:

- `alfvenwave*`: 1-D Cartesian tests for an Alfvén wave in each of the 1-, 2-, and 3-directions for each of the `trnvrsn=0,1` algorithms using `CMoC` and, for `trnvrsn=0` only, each of the `HSMoC`, `MoC`, and original `CT` algorithms. The latter is included to demonstrate the catastrophic results of trying to transport an Alfvén wave without proper upwinding.

- `shearaw*`: 2-D Cartesian tests for a shear Alfvén pulse in each of the 1-2, 2-3, and 3-1 planes and for each of `trnvrsn=0,1`. As with advection, proper transport of Alfvén waves in 1-D did not translate to stable transport in 2-D until the introduction of FIT (`trnvrsn=1`) in version 3.6.

- `advectaw*`: Similar to the previous tests with a Galilean transformation added. This test further demonstrates the superiority of FIT (`trnvrsn=1`) over legacy transport (`trnvrsn=0`) in 2-D dynamics. I hasten to add, however, that for real problems with Alfvén waves moving in all direction at all times, there is no discernible difference between legacy and FI transport! One can take these test results far too seriously!

4.3.3 Shock-tube tests

A "shock-tube" is a 1-D "Riemann problem" consisting of two constant and steady states which come in contact and interact with each other at $t = 0$. Such tests are useful since they engage the full dynamics of MHD and their analytical solutions are known. They are limited, however, since for the most part, they test the code in only one physical dimension

at a time. Still, they are the traditional way of determining the reliability of fluid codes, and comprise nearly half of the individual tests in the *ZEUS* test suite.

In this release, their are around 100 shock tube test problems, including:

- `ryujones*`: separate tests for each of the twelve problems described in Ryu & Jones (1995, ApJ, 422, 228) done in each of the 1-, 2-, and 3-directions for each of `trnvrsn=0, 1`. Two of the problems are also used to test the isothermal option `ISO`, an experimental conservative-in-momentum transport algorithm (`trnsprt=2`), and two pre-`CMoC` magnetic induction algorithms, `HSMoC` and `MoC`.

- `briowu*`: the Brio & Wu (1988, J. Comput. Phys., 75, 400) compound wave shock-tube in each of the 1-, 2-, and 3-directions for each of `trnvrsn=0,1,2`

- `sod`: the classic Sod (1978, J. Comput. Phys., 27, 1) shock-tube (hydrodynamical) in just the 1-direction for each of `trnvrsn=0,1`

- `jonshktb*`: a shock-tube problem devised by Jon Ramsey that seems to demonstrate a weakness in the total energy algorithm in `dzeus36` (cause still unknown at this writing)

- `reflecting*`: uses a shock-tube similar to Ryu & Jones' test 2a (`ryujones_fig2a-x1`) but with $B_1 = 0$ to test the reflecting boundary conditions. The two problems (one with reflecting BC at the outer 1-boundary and only half the grid, the other with no reflecting BC and with the entire grid) should be, and are, identical to machine round-off error.

- `twoshocks*`: sets up a shock-tube with two discontinuities (three constant states) to allow two shocks to pass through each other. For this problem there is a partial analytical solution, which the test passes.

- `shkset2d`: sets up Ryu & Jones' test 1a on a 2-D grid with the discontinuity oriented with its normal at an angle $\tan^{-1}(\frac{1}{2})$ relative to the 1-direction. This tests full dynamics and discretisation errors in 2-D, as well as "slip periodic boundary conditions", a new feature to v3.6.

4.3.4 Other 1-D tests

- `cshock*`: both the isothermal and adiabatic continuous shock ("C-shock") solutions presented in Duffin and Pudritz (2008, MNRAS, 391, 1659) to test the one-fluid ambipolar diffusion algorithm. Three methods of computing the time step are also tested: regular CFL (`iscyad=0`), subcycling (`iscyad=1`), and "super-stepping" (`iscyad=2`), the latter from Meyer, Balsara, and Aslam (2012, MNRAS, 422, 2102).

- `soundwave*`: full 1-D hydrodynamic initialising a small (1 : 6000) square-pulse pressure perturbation, launching sound waves in both directions to test diffusion of subsonic disturbances (substantial) and propagation speed (c_s to within less than 0.1%)

- `zpinch*`: the classic "Bennett pinch" problem used to test *ZEUS*' ability to "hold onto" a steady state equilibrium solution in cylindrical coordinates.

- `mhdrot*`: magnetic braking test from Mouschovias & Paleologou (1980, ApJ, 237, 877) and used to test *ZEUS-2D* by Stone & Norman (ApJS, 1992, 80, 791)

- `sedov-x1*`: the spherically symmetric "Sedov blast wave" (Sedov, 1948, J. Appl. Math. and Mech., 10, 241) used as a 1-D test for the internal and total energy equations (`itote=0,1`) in spherical polar coordinates. See also §4.3.6.

4.3.5 Other 2-D tests

- `vortex*`: transport of a vortex ring (a circulating ring of hydrodynamical gas) across a 2-D grid in each of the 1-2, 2-3, and 3-1 planes using each of `trnvrsn=0,1`. As with other such problems, Legacy Transport reduces the ring into a series of deep striations perpendicular to the flow direction. Conversely, FIT does an acceptable job. Still, as far as I know, no evidence of "legacy striations" has ever been noted in "real simulations".

- `mhdblast*`: 2-D planar-symmetric MHD blast wave used by Gardiner & Stone (2005, Commput. Phys. Comm., 89, 127) to test full MHD dynamics in 2-D; it has no known analytic solution. This test is performed in each of the 1-2, 2-3, and 3-1 planes with each of `trnvrsn=0,1,2`.

- `orszagtang*`: 2-D MHD vortex ring first reported in Orszag & Tang (1979, J. Fluid Mech., 90, 129) to test full MHD dynamics in 2-D; it has no known analytic solution. This test is performed in each of the 1-2, 2-3, and 3-1 planes with each of `trnvrsn=0,1,2`. It is also used to demonstrate that periodic boundary conditions are good to machine round-off error in all three planes and for both `trnvrsn=0,1`.

- `khinst*`: the classic $M = 2$ hydrodynamical Kelvin-Helmholtz instability done only in the 1-2 plane for both `trnvrsn=0.1`.

- `corona*`: scaled-back test version of the simulation discussed in Ouyed & Pudritz (1997, ApJ, 484, 794) of a protostellar jet launched from a Keplerian accretion disc maintained as a boundary condition. It tests full MHD dynamics using both `trnvrsn=0,1` and two types of boundary conditions (`oork=1,2`) in cylindrical coordinates, and has no known analytic solution.

- `rotor*`: uniformly rotating toroid threaded by a uniform axial magnetic field located a) in the grid centre, b) at the inner boundary, and c) at the outer boundary. All should (and do) give identical results and agree with the analytical solution. This was designed to test the ability of the boundary conditions to launch a shear Alfvén wave in cylindrical coordinates.

4.3.6 3-D tests

- `sedov-*`: sets off the Sedov blast wave (see §4.3.4) in a) a full 3-D Cartesian grid, and b) in just one octant of the 3-D grid with reflecting boundary conditions. This problem has a known analytical solution, and tests full hydrodynamical dynamics as well as the accuracy of the reflecting boundary conditions (good to machine round-off error).

- `supaturb*`: scaled-back test version of the signature simulation of `CMoC`; driven, super-Alfvénic turbulence. This problem has no known analytical solution but has some analytical predictions that can be used to test isothermal MHD dynamics in 3-D using both `trnvrsn=0,1`.

4.3.7 Adding tests to the test suite

There are two things the user must do to add tests to the test suite.

1. A copy of working `dzeus36.s` and `zeus36.mac` files must be added to one of the directories `1d`, `2d`, or `3d` in `testz/testsuite` using a unique name that describes the new test. Thus, if the name of a new 1-D test is `usersnew1dtest`, copy its working `dzeus36.s` file to `testz/testsuite/1d/usersnew1dtest.s` and its working `zeus36.mac` file to `testz/testsuite/1d/usersnew1dtest.mac`.

2. Open up the master key file (`testsuite/testz_key`) in a text editor and insert a line resembling:

   ```
   usersnew1dtest <id> 1 <nhy> <last dt> s
   ```

   (without the angle brackets `<>`), where:

   - `id` is a two-character problem tag different from all other problem tags already used in the master key file;
   - `nhy` is the number of MHD time steps taken by the new test when run properly (reported on screen by *ZEUS-3D* when the job is finished); and
   - `last dt` is the last time step to 16 significant figures taken by the new test when run properly (also reported on screen by *ZEUS-3D* when the job is finished).

   Only one space between each entry is needed, though more can be added if desired. The first time the test suite is run and saved with the new tests included in the master key file, the entries in the new master key file will appear with the same spacing as the other tests.

   Note that with the trailing "s", the new tests will be run whether `xtz10 run` or `xtz10 rerun` is typed just after the new tests have been added.

Finally, to minimise the number of times *ZEUS-3D* has to be recompiled throughout execution of the testsuite, the user should try to find an existing test with grid dimensions in its `.s` file and *EDITOR* macro settings in its `.mac` file suitable for the new test problem. There may be none, in which case the `.s` and `.mac` files are what they are and the new test can be added anywhere in the master key file the user likes. However, if an existing test is found with suitable grid dimensions and macro settings, these should be duplicated in the `.s` and `.mac` files of the new test, and then in the master key file the new test should be listed immediately before or after the similar existing test. In this way, *ZEUS-3D* will not be recompiled because of the new test, and the testsuite will run through all that much faster.

4.4 Output from *TESTZ*

TESTZ generates two ascii output files.

1. `testz_key.new` lists the latest set of test problems by name, along with an assigned problem tag, its dimensionality, values of the two key comparators from the last run of the test suite, and a flag to indicate whether this test was run ("`r`") or skipped ("`s`"). This file overwrites the "master key file" `testsuite/testz_key` when `xtz10 savekey` or `xtz10 saveall` is issued (see §4.2.3).

2. `testz_log` contains a copy of all output to the screen, including which tests were run or skipped, whether or not key comparators matched previous values, and some scant information should any of the tests fail to run.

4.5 Namelist parameters for *TESTZ*

There is only one namelist for *TESTZ*, namely `tzcon`. For a reminder of the nature of namelists and how they are used in the *ZEUS* family of codes, the reader is referred to the first page of Appendix B in the `dzeus36` user manual.

`TZCON`—`TestZ CON`trol (defined in subroutine `nmlsts`)

This namelist controls what tests from the test suite are to be performed, whether their output is to be saved, whether the master key file is to be updated, *etc.* To specify which tests are done, each element of "`tests`" (the first namelist parameter listed below) can be filled with a specific test name (as listed in the file "`testsuite/testz_key`"), and/or one can use a select number of "wildcards":

"`*`"	⇒	do all tests
"`ry*`"	⇒	do all tests whose name begins with "`ry`"
"`1d`"	⇒	do all 1-D tests
"`2d`"	⇒	do all 2-D tests
"`3d`"	⇒	do all 3-D tests

Notes:

1. The syntax is very limited and specific. Thus, one cannot specify `*x1*` to get all tests with the characters "`x1`" somewhere in their name.

2. Setting more than one element of "`tests`" results in the union of all tests specified being done, not the intersection. Thus, setting:

   ```
   tests='ry*','2d'
   ```

   does all tests whose name begins with "`ry`", as well as all 2-D tests.

```
parameter                    description                        default

tests   (ntst)  (character*40) specifies which tests to perform.    '*'
skonly          0 => do all tests listed in "tests"                 0
                1 => do only the skipped tests listed in "tests"
iotty           logical unit for standard io (screen)               6
                0 => no output to screen, otherwise iotty = 6
iolog           logical unit for log file                          10
                0 => no output to logfile, otherwise iolog = 10
tol             real*8; tolerance for declaring "last dt" to be      1.0d-7
                a match.
verbose         0 => suppresses screen output from xdzeus36          0
                1 => all screen output from xdzeus36 written
                     expect many pages!)
compiler        used to compile dzeus36                            'gfortran'
rors            1 => run all "tests", doing nothing with             1
                     generated files
                2 => save testz_key.new file generated from most
                     recent testz run, and manage io files if
                     they are to be kept (keepio)
                3 => do both; run all tests & save/manage files
keepio          0 => discard all .ps and .mp4 files                  0
                1 => move .ps, .mp4 files generated by tests to
                     directory <testdir>/archive/dumps.<date>
                2 => same as 1, then compress directory into the
                     tarball <testdir>/archive/dumps.<date>.tar

       namelist / tzcon    /
     1               tests   , skonly  , iotty   , iolog   , tol
     2             , verbose , compiler, rors    , keepio
```